

At production sample sizes, read the statistic, not the p-value

n = 20,000 vs 5,000

p ~ 1e-186 for a trivial wiggle

KS statistic 0.23: a real, visible 23-point gap

The p-value twitches. The statistic measures.

Calibrate the statistic against the feature's normal wobble. Ignore the p-value.

Figure: at $n = 20,000$ vs $5,000$, even a trivial wiggle returns $p \sim 1e-186$. The KS statistic 0.23 shows the real 23-point gap. Read the statistic, not the p-value.

OCTYPE html >

Contents

ML Engineer Crash Pack	2
How to use this pack	3
Part 1. Decision rules	5
1.1 The baseline ladder	5
1.2 The threshold rule: set it from the cost ratio	9
1.3 Promotion gates: from run to production	13
1.4 Shadow vs canary: what each one buys	16
1.5 Retraining triggers	18
1.6 Cost-per-prediction budgets	21
1.7 The iteration discipline	22
Part 2. Numbers and formulas to memorize	25
2.1 PSI: the drift dial	25
2.2 KS test: reading the statistic and the p-value	27
2.3 Chi-square for categorical drift	29
2.4 Confusion matrix arithmetic	31
2.5 AUC vs the business metric: the gap	34

2.6 Canary math: detection speed vs blast radius	35
2.7 Distillation numbers	37
2.8 GBDT vs MLP on tabular data: the measured trade	38
2.9 The leakage shape: offline 0.94, production 0.61	40
2.10 The four monitoring layers	41
Part 3. Classic traps	44
Trap 1. Training/serving skew	44
Trap 2. Point-in-time leakage	45
Trap 3. Optimizing AUC while the business metric flatlines	46
Trap 4. Silent feature drift	47
Trap 5. Two changes at once	47
Trap 6. Test-set peeking	47
Trap 7. Tuning until the complex model wins by luck	47
Trap 8. Rollback theater	48
Trap 9. Prediction drift without Layer 1	50
Trap 10. The short shadow and the comparison-free canary	51
Part 4. Rapid-fire Q&A	52
ML lifecycle	52
Feature pipelines	54
Experiment tracking	56
CI/CD for ML	57
Monitoring and drift	59
Classical ML	61
Cost-aware ML	63
Part 5. One-page cheat sheet: 30 numbers and rules	65

ML Engineer Crash Pack

How to use this pack

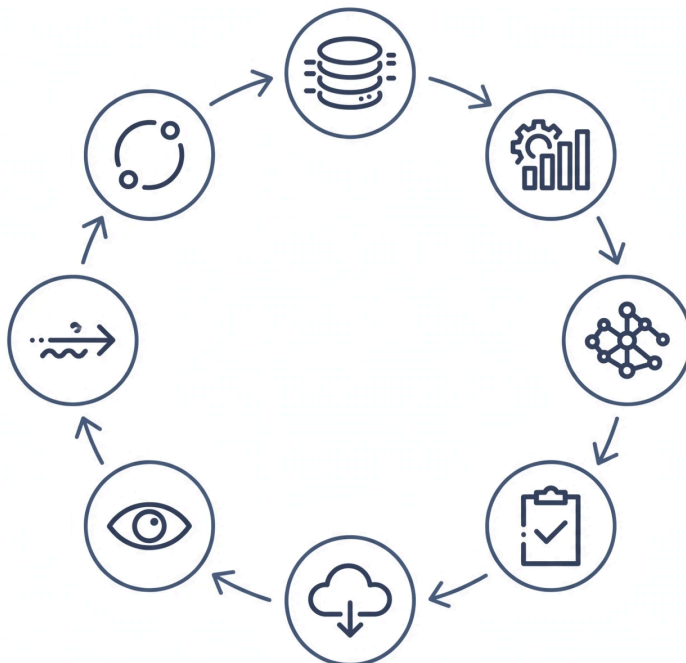
This is the condensed field volume for Role Track 9, the ML engineer: the person who owns machine learning models in production, end to end. The data that feeds them, the training that builds them, the services that serve them, and the monitoring that keeps them honest.

Read it in one sitting, in order. Part 1 gives you the decision rules that replace guessing. Part 2 gives you the numbers and formulas to memorize, each with units and meaning. Part 3 is the failure catalog: the traps that end production models, with the shape of each bug. Part 4 is rapid-fire Q&A with worked answers. Part 5 is the one-page cheat sheet: 30 numbers and rules that matter most.

Every number marked as measured comes from code executed on this machine during writing. Worked examples use illustrative prices and synthetic data where noted.

Key takeaways

- The ML engineer asks not "what is the best model we can build" but "what is the best model we can run daily, without surprises."
- A large share of production ML failures are data failures wearing a model costume.
- The loop is data, training, serving, monitoring, and back to data. Nothing on it is a one-time step.



How to read this diagram

1. Data pipelines feed training pipelines. Compute features identically at train time and serve time. Any gap here is training/serving skew.
2. Training pipelines produce model artifacts. Versioned, logged, and reproducible.
3. Serving systems expose the artifact, with rollout discipline: shadow, canary, full.
4. Monitoring closes the loop. Input drift, prediction drift, and business-metric decay all route back to data or training.
5. The business metric both measures the monitoring stage and constrains the serving stage. Latency and cost budgets come from the product, not from the model.

Part 1. Decision rules

The chapters ahead follow the ML lifecycle in order: decisions first, then the numbers behind them, then the traps, then drills. This part is the decisions: seven rules that replace judgment calls with procedure, from the baseline ladder to the iteration discipline.

These are the rules that replace judgment calls with procedure. When the rule and your gut disagree, follow the rule and write down why your gut objected. That note is how the rules get better.

1.1 The baseline ladder

Step 1: the rule.

Never start with the model you hope to ship. Climb the ladder in order. Each rung is cheap, and each rung catches a different class of mistake. Each rung must beat the last on the holdout set (data the model never trained on; the final judge) by a margin that survives the statistics. "It felt better in the notebook" is not a rung.

Step 2: the cheap rungs: rule, constant.

A rule. Hand-written, from domain sense: "flag customers with 3+ support calls or 2+ late payments." It takes an hour and anyone can audit it. Rules catch framing errors: if the rule is competitive with your model, the problem was simpler than you thought. **A constant.** Predict the majority class, or the base rate (the overall fraction of positives in the data) as a probability. This is the floor. If a trained model cannot beat "always predict no churn," the pipeline has the bug, not the model.

Rung 1: hand rule
catches framing errors



Rung 2: constant
catches broken pipelines



Rung 3: logistic regression
is the signal linear?



Rung 4: gradient-boosted trees
wins most tabular fights



Rung 5: deep learning
only with evidence

How to read this diagram

1. Read bottom-up by cost: each rung is more expensive than the last in training time, serving cost, and debugging surface.
2. The green rungs are always worth the hour they cost. The yellow rung (GBDT) is the default stopping point for tabular data.
3. The red rung (deep learning) needs a written justification: the measured margin over rung 4, and the serving budget it fits in.
4. The ladder also runs in reverse as a diagnostic. If rung 3 beats rung 4, your features are mostly linear or someone misconfigured your tree. If rung 2 beats rung 3, you have a pipeline bug.

Step 3: the working rungs: linear, GBDT.

A linear model. Logistic regression on simple features. Fast, calibrated, and a strong signal about whether the problem is mostly linear. **A gradient-boosted tree.** Often the last step before deep learning, and often the last step, period. On tabular data this rung wins most fights.

The ladder doing its job, measured: on a churn dataset (20,000 customers, 17.8 percent base churn rate), the hand rule reached precision 0.284 (of its flags, 28.4% were real churners) and recall 0.363 (it caught 36.3% of the real churners). Logistic regression reached AUC 0.7065 on the holdout set (AUC measures ranking quality: how often a random churner scores above a random non-churner; Section 2.5). Gradient-boosted trees reached AUC 0.6926, slightly worse. The fancier model lost. The signal was mostly linear, and the honest move was to ship the linear model, not to tune the tree until it won by luck.

Step 4: the top rung, and the verdict.

Deep learning. Only after the ladder, and only with evidence it beats rung 4. Keep tuning the complex model until it beats the simple one on the validation set, and you overfit with extra steps. (Validation set: data used to compare models during development, separate from training and the final test set.) Overfitting means memorizing the tuning data's noise instead of its pattern. The ladder's verdict is final for this round: write it down, move to the next bottleneck.

The baseline ladder tells you which model to try first. The next decision is where to cut its scores: the threshold rule, set from the cost ratio of mistakes, never from habit.

\n

The baseline ladder: climb in order, never skip

Read bottom-up by cost. Each rung must beat the last on the holdout set.

rung 5

● deep learning

Ships only with a measured margin over rung 4 and a serving budget it fits in.

needs justification

rung 4

● gradient-boosted trees (GBDT)

On tabular data this rung wins most fights. Often the last step, period.

default stop

rung 3

● logistic regression on simple features

Tells you whether the problem is mostly linear.

fast, calibrated

rung 2

● hand rule: 3+ support calls or 2+ late payments

Catches framing errors: if the rule is competitive, the problem was simpler than you thought.

1 hour

rung 1

● constant: always predict the base rate

If a trained model cannot beat this, the pipeline is broken.

the floor

Cost grows upward: training time, serving cost, and debugging surface. The ladder also runs in reverse as a diagnostic: if rung 3 beats rung 4, your features are mostly linear or the tree is misconfigured. If rung 2 beats rung 3, you have a pipeline bug.

Figure: the baseline ladder. Climb in order and never skip. A trained model that cannot beat the constant base-rate predictor means the pipeline has the bug, not the model. *Source: generated for this pack.*

How to read this diagram

1. The constant predictor is the floor. It costs nothing and sets the number every trained model must beat.
2. The one-hour hand rule catches framing errors: if a hand rule is competitive, the problem was simpler than you thought.
3. Logistic regression tells you whether the problem is mostly linear; GBDT is usually the last stop before deep learning.
4. If a trained model cannot beat the constant, debug the pipeline, not the model.

1.2 The threshold rule: set it from the cost ratio

Step 1: a score is not a decision.

The threshold converts $P(\text{churn})$ into an action. A model with great AUC and a threshold picked at 0.5 by habit can still lose money. Ranking quality and the cut are different things.

Step 2: the cost math.

$$\begin{aligned} p * C_{fn} &> (1 - p) * C_{fp} \\ p &> C_{fp} / (C_{fp} + C_{fn}) \end{aligned}$$

p is the predicted probability, C_{fn} the cost of a false negative, C_{fp} the cost of a false positive. Flag the customer when the expected cost of flagging is lower than the expected cost of staying silent. Worked: a saved customer is worth \$200 (C_{fn}), a retention offer costs \$20 (C_{fp}). The threshold is $20 / (20 + 200) = 0.0909$. Flag anyone whose churn probability tops about 9 percent, not 50 percent.

$$p^* = C_{fp} / (C_{fp} + C_{fn})$$

where: p^* means the decision threshold on the predicted probability. C_{fp} means the cost of a false positive. C_{fn} means the cost of a false negative. Flag when $P(\text{churn}) > p^*$.

The threshold is a price tag, not a habit

Flag when $P(\text{churn}) > p^*$

$$p^* = C_{fp} / (C_{fp} + C_{fn})$$

C_{fp} : cost of a false positive. C_{fn} : cost of a false negative.

Cheap false positives push p^* down: flag more.

Expensive false positives push p^* up: flag less.

0.5 is a habit. p^* is arithmetic on the cost of mistakes.

Figure: the decision rule. Flag when $P(\text{churn})$ tops p^* , where $p^* = C_{fp} / (C_{fp} + C_{fn})$. Cheap false positives push p^* down; expensive ones push it up.

Worked example. A saved customer is worth \$200 (C_{fn}); a retention offer costs \$20 (C_{fp}).
 $p^* = 20 / (20 + 200) = 20/220 = 0.0909$, **about 9%**. Flag anyone whose churn probability tops 9 percent, not the habitual 50 percent. Question 1 works the same setup.

Step 3: same scores, three businesses.

The code below shows profit moving while the ranking quality stands still. Same scores, three thresholds, three very different businesses.

```

import numpy as np

# WHAT: pick the decision threshold from the error-cost ratio, not fr
# WHY: the threshold turns a score into an action, and the right acti
#   on what each mistake costs. This is a product decision expressed
# WHAT BREAKS: costs drift. Recompute the ratio when pricing or the o
C_fn, C_fp = 200.0, 20.0          # $200 saved customer, $20 retentio
t_star = C_fp / (C_fp + C_fn)    # flag when p > C_fp / (C_fp + C_fn
print("optimal threshold:", round(t_star, 4))  # 0.0909, not 0.5

# WHAT: profit at three thresholds on the same scored population.
# WHY: ranking quality (AUC) is identical in all three rows. Only the
#   moves, and the cut is what the business feels. Optimizing AUC alo
#   cannot find the cut; only the cost math can.
# WHAT BREAKS: the exact argmax comes from sweeping YOUR validation s
#   calibrated probabilities. Calibrated means the scores match real
#   the number you ship.
rng = np.random.default_rng(11)
n = 20000
y = (rng.random(n) < 0.18).astype(int)          # 18% base churn
s = np.clip(rng.normal(0.18, 0.12, n) + y * 0.25, 0.001, 0.999) # a

def profit(y, s, t):
    # Net dollars vs doing nothing: saved customers minus wasted offe
    flag = s >= t
    tp = np.sum(flag & (y == 1))
    fp = np.sum(flag & (y == 0))
    fn = np.sum(~flag & (y == 1))
    return tp * C_fn - fp * C_fp - fn * C_fn

for t in [0.5, t_star, 0.25]:
    print(f"threshold {t:.4f} -> profit ${profit(y, s, t):,.0f}")
# threshold 0.5000 -> profit $-307,080    (the 0.5 habit loses money)
# threshold 0.0909 -> profit $470,720    (the cost-ratio cut)
# threshold 0.2500 -> profit $535,960    (validation sweep finds the

```

The 0.5 habit loses \$307,080 on this population

Same model, same AUC, three businesses

0.5 habit

-\$307,080

validation sweep

+ \$535,960

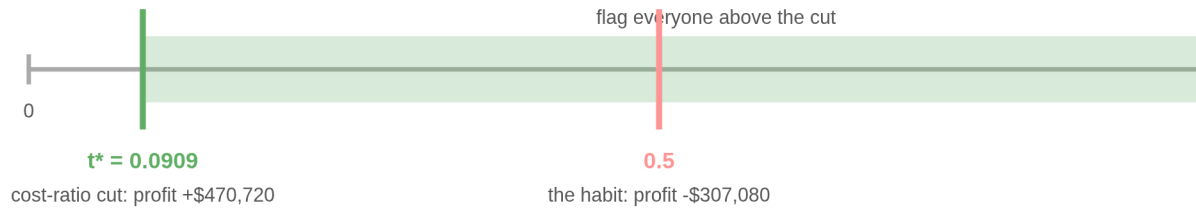
The AUC never moved. The business moved by \$777,800 on the cut a

Figure: threshold economics. Same model, same AUC, three businesses. The 0.5 habit loses \$307,080; the cost-ratio cut makes \$470,720; the validation sweep finds \$535,960.

. The cost-ratio threshold makes \$470,720. The validation sweep finds \$535,960. Same model, same AUC, three businesses.

The threshold is a price tag, not a convention

Churn score from 0 to 1. The cut sits where flagging costs less than staying silent: $t^* = C_{fp} / (C_{fp} + C_{fn}) = 20 / 220 = 0.0909$.



Same model, same AUC. Only the cut moves, and the cut is what the business feels.

Figure: the threshold is a price tag. The cost-ratio cut at 0.0909 flags everyone whose churn probability tops about 9 percent and makes \$470,720. The 0.5 habit flags only above 50 percent and loses \$307,080. *Source: generated for this pack.*

Step 4: the price tag.

Cost of one false negative (missed fraud): \$85

Cost of one false positive (manual review): \$3

$$\begin{aligned} \text{threshold} &= \text{FP-cost} / (\text{FP-cost} + \text{FN-cost}) \\ &= 3 / (3 + 85) \\ &= 0.034 \end{aligned}$$

Not 0.5. The threshold is a price tag, not a convention.

1.3 Promotion gates: from run to production

Step 1: stages, not vibes.

A model registry tracks artifacts through stages: none, staging, production, archived. Gates control movement between stages. The gates are the difference between "we deployed a model" and "we have a deployment process."

Step 2: the machine gates.

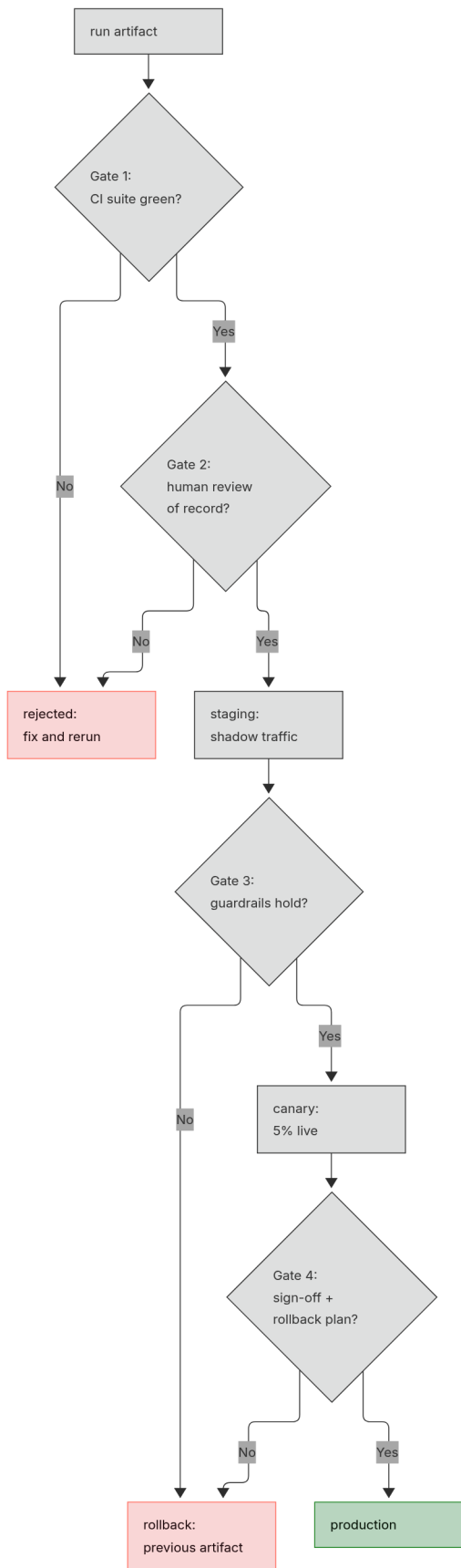
Gate 1: automated checks. The candidate artifact must pass the CI suite: data tests, model tests, and the golden-set performance floor. No human needed, no exceptions granted. **Gate 2: offline review.** A second engineer reads the experiment record: the hypothesis, the comparison table, the leakage check. It catches what automation cannot see.

Step 3: the reality gates.

Gate 3: shadow then canary. The model serves real traffic without affecting users (shadow), then a small slice of live traffic (canary), with guardrail metrics at each step. Section 1.4 walks the ladder.

Step 4: the human gate.

Gate 4: sign-off and rollback plan. A named human approves. The plan is only real if restoring the previous artifact takes minutes, not hours.



How to read this diagram

1. Every arrow forward has a named gate. Every gate has a defined failure path. Note the rollback arrow points at the previous artifact, never at "fix forward."
2. Gate 1 is machines. Gate 2 is a second pair of eyes for thirty minutes. Skipping gate 2 is how test-set overlap ships.
3. Gate 3 has two stages because shadow and canary answer different questions (Section 1.4).
4. Gate 4 names a human and demands a one-command rollback. A rollback that takes an hour is not a rollback plan; it is a hope.

Promotion gates decide when a run becomes a production candidate. The next question is how to expose it to reality safely: shadow vs canary, and what each one buys you.

1.4 Shadow vs canary: what each one buys

Step 1: shadow: the code meets reality, users untouched.

The rollout ladder is shadow, then canary, then full. Each stage buys information at a price; skipping a stage means paying its price later, with interest. **Shadow** is observe-only: the pipeline duplicates requests, production serves the user, the shadow model scores a copy, and the pipeline logs both predictions. Nothing the shadow model says reaches a customer. Shadow answers what offline eval cannot: does the candidate handle the real input distribution, including the malformed rows the training set never had? Does it meet latency under real concurrency (many requests arriving at once)? Do its predictions disagree with the production model in a pattern that suggests a bug? Run shadow for at least one full business cycle: a week for weekly-seasonal traffic, a month for monthly. Shorter runs miss the weekend, the payday, the holiday.

Step 2: canary: a small live slice, watched hard.

The candidate serves typically 1 to 5 percent of live traffic. The team watches guardrail metrics against the production baseline. Three kinds of guardrails. **Business:** the metric the model serves (conversion, fraud loss) must not regress beyond tolerance. **Model:** prediction distribution and calibration must stay within bounds. **System:** latency (p99: the latency that 99% of requests beat), error rate, and cost per prediction must hold.

Step 3: the decision, as code.

One tripped guardrail holds the canary for human review. Two or more roll back immediately: correlated failures are never coincidence.


```

# WHAT: the canary decision, as code. Compares guardrail metrics betw
#   canary slice and the production baseline over the canary window.
# WHY: "looks fine on the dashboard" is not a decision procedure. Cod
#   make the promote/hold/rollback call consistent at 3 a.m.
# WHAT BREAKS: short windows lie. A 2-hour canary cannot see daily se
#   Minimum window is one full business cycle.
def canary_decision(canary, baseline):
    # Each metric: (canary_value, baseline_value, tolerance, worse_di
    # Tolerances come from the metric's normal wobble, measured on hi
    # so the canary is judged against reality, not against zero.
    guardrails = {
        "conversion_rate": (canary["conv"], baseline["conv"], 0.02, "
        "p99_latency_ms": (canary["p99"], baseline["p99"], 0.15, "hig
        "error_rate": (canary["err"], baseline["err"], 0.50, "higher"
        "cost_per_1k": (canary["cost"], baseline["cost"], 0.10, "high
    }
    verdicts = {}
    for name, (c, b, tol, worse) in guardrails.items():
        rel = (c - b) / b if b else 0.0      # relative change vs ba
        tripped = (rel < -tol) if worse == "lower" else (rel > tol)
        verdicts[name] = ("TRIPPED", round(rel, 3)) if tripped else (
    tripped = [n for n, (v, _) in verdicts.items() if v == "TRIPPED"]
    if len(tripped) >= 2:
        return "ROLLBACK", verdicts
    if len(tripped) == 1:
        return "HOLD_FOR_REVIEW", verdicts
    return "PROMOTE", verdicts

base = {"conv": 0.050, "p99": 120.0, "err": 0.010, "cost": 0.083}
healthy = {"conv": 0.0505, "p99": 126.0, "err": 0.010, "cost": 0.086}
broken = {"conv": 0.0495, "p99": 150.0, "err": 0.011, "cost": 0.098}
print("healthy canary ->", canary_decision(healthy, base)[0]) # PROM
print("degraded canary ->", canary_decision(broken, base)[0]) # ROL
# degraded trips p99 latency (+25% vs 15% tolerance) and cost (+18% v

```

Step 4: the full ladder.

```

baseline (100%) → shadow copy (0% live traffic, logs only)
    |
    ▼ pass quality gates
baseline (95%) + canary (5%) → compare guardrails for days
    |
    ▼ no guardrail trips
baseline (50%) + candidate (50%) → A/B test on the business metric
    |
    ▼ wins
candidate → 100%

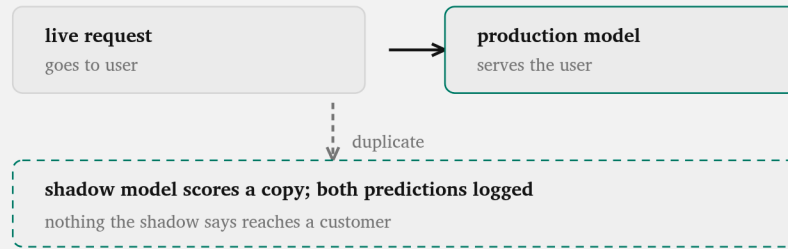
Shadow tests the code. Canary tests the world. A/B tests the money.

```

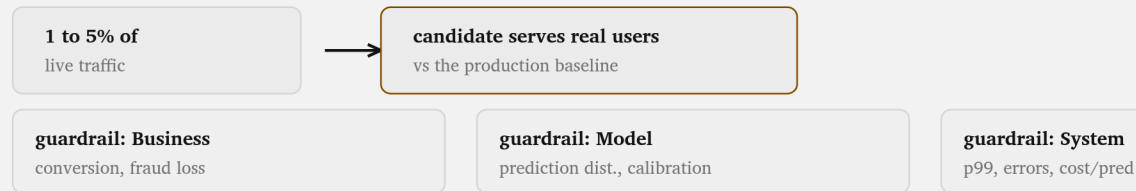
Shadow vs canary: what each stage buys

Shadow asks: does it survive reality? Canary asks: does the business metric hold?

SHADOW: observe only, users untouched



CANARY: a small live slice, watched hard



Decision, as code: one tripped guardrail holds for human review. Two or more roll back immediately: correlated failures are never coincidence. Shadow runs at least one full business cycle; a canary with no baseline comparison is just a slow full rollout.

Figure: shadow vs canary. Shadow observes without touching users; a canary serves a small live slice watched against guardrails. *Source: generated for this pack.*

How to read this diagram

1. Shadow: the candidate scores a copy of live traffic, but nothing it says reaches a customer.
2. Canary: 1 to 5 percent of live traffic goes to the candidate, watched against guardrails on business metrics.
3. Shadow buys safety, canary buys evidence. The order is shadow first, then canary, then full rollout.

1.5 Retraining triggers

Step 1: the principle.

Retrain when the world has moved enough that the current artifact is the wrong model for the current data.

PSI above 0.25 on a key feature → trigger retrain
 accuracy down 2 points → trigger retrain
 30 days elapsed → scheduled retrain

Retraining is a fire alarm, not a calendar habit
 (unless the calendar turns out cheaper than the alarm).

Step 2: the three trigger styles.

Scheduled. Retrain every N days regardless

Three retrain triggers

SCHEDULED

every N days, N from measured decay

THRESHOLD

PSI above the line K windows running

EVENT-DRIVEN

launch, new market, schema change

Usually the team combines all three.

Figure: the three retrain triggers. Scheduled every N days from measured decay; threshold when a monitor trips K windows running; event-driven on world changes.

. Simple, predictable, the right default for slowly changing domains. Set N from measured decay: if AUC drops 1 point per month, monthly retraining holds the line. **Threshold.** Retrain when a monitor trips: PSI (Population Stability Index: a score that measures how far a feature's serving distribution has moved from its training distribution; Section 2.1 works it fully) above the per-feature line for K consecutive windows, or the business metric down by the agreed amount. Thresholds avoid

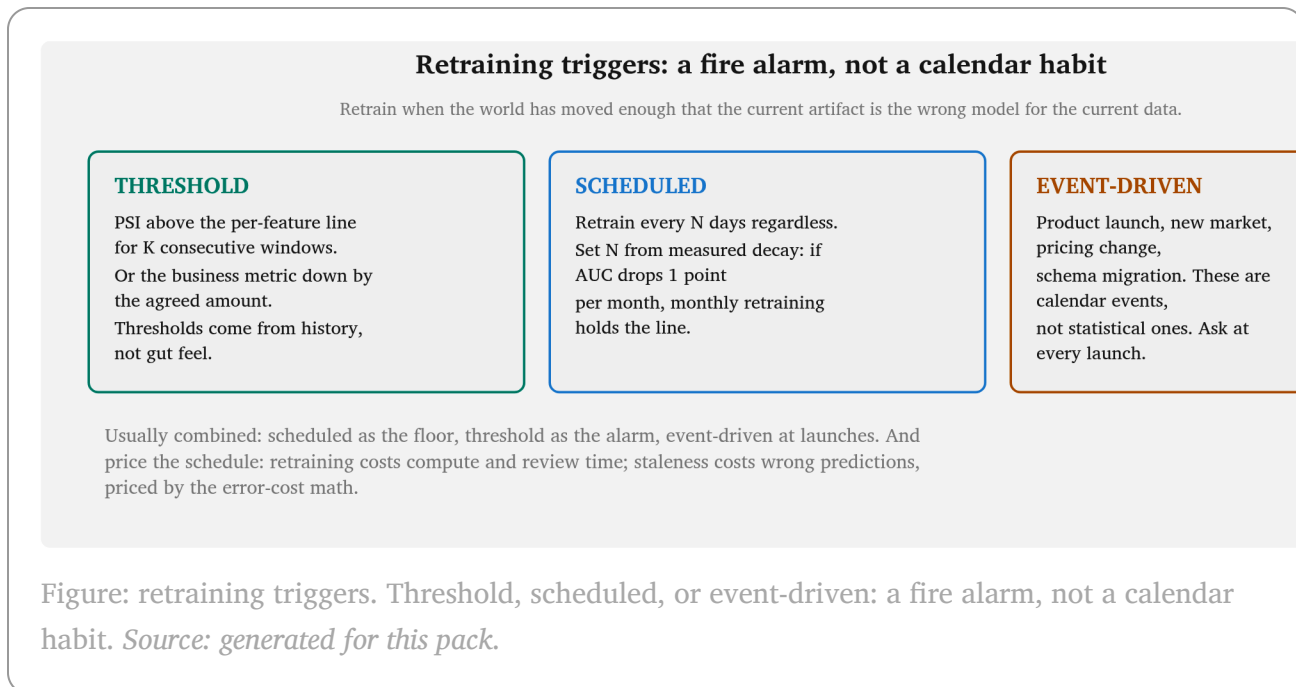
retraining on noise, but they need tuning discipline: thresholds come from history, not gut feel.

Event-driven. Retrain on known world changes: a product launch, a new market, a pricing change, a schema migration. These are calendar events, not statistical ones; the launch checklist asks "does this launch invalidate any model?" Usually the team combines all three.

Step 3: the retrain is a pipeline run, not a heroic effort.

The fresh extract passes the data tests. Training runs the tracked config. The candidate clears the promotion gates. The rollout follows the shadow-canary ladder. If retraining requires a hero, the pipeline is the thing to fix. And price the schedule: retraining costs compute and review time; staleness costs wrong predictions, priced by the error-cost math from Section 1.2. When staleness is cheap and retraining is expensive, schedule loosely. When a stale model burns money daily, trigger tightly.

Retraining triggers tell you when to rebuild the model. The next question is what each prediction costs while it serves: the unit economics of the serving stage.



How to read this diagram

1. Threshold: PSI above the per-feature line for K consecutive windows, or the business metric down by the agreed amount.
2. Scheduled: retrain every N days, with N set from measured decay, not gut feel.
3. Event-driven: product launch, new market, schema change. Thresholds come from history.

1.6 Cost-per-prediction budgets

Step 1: the unit.

The unit to optimize is cost per prediction at the required quality, not cost per server or cost per month. Servers are not the product; predictions are.

Step 2: the formula.

$$\text{cost per 1k predictions} = (\text{hourly serving cost}) / (\text{predictions per hour})$$

Measure throughput (predictions completed per hour) AT the latency SLO (service-level objective: the latency promise the product makes). Quoting a batch throughput number for a real-time SLO inflates the denominator and lies.

Step 3: the worked sketch.

Illustrative prices, not vendor quotes:

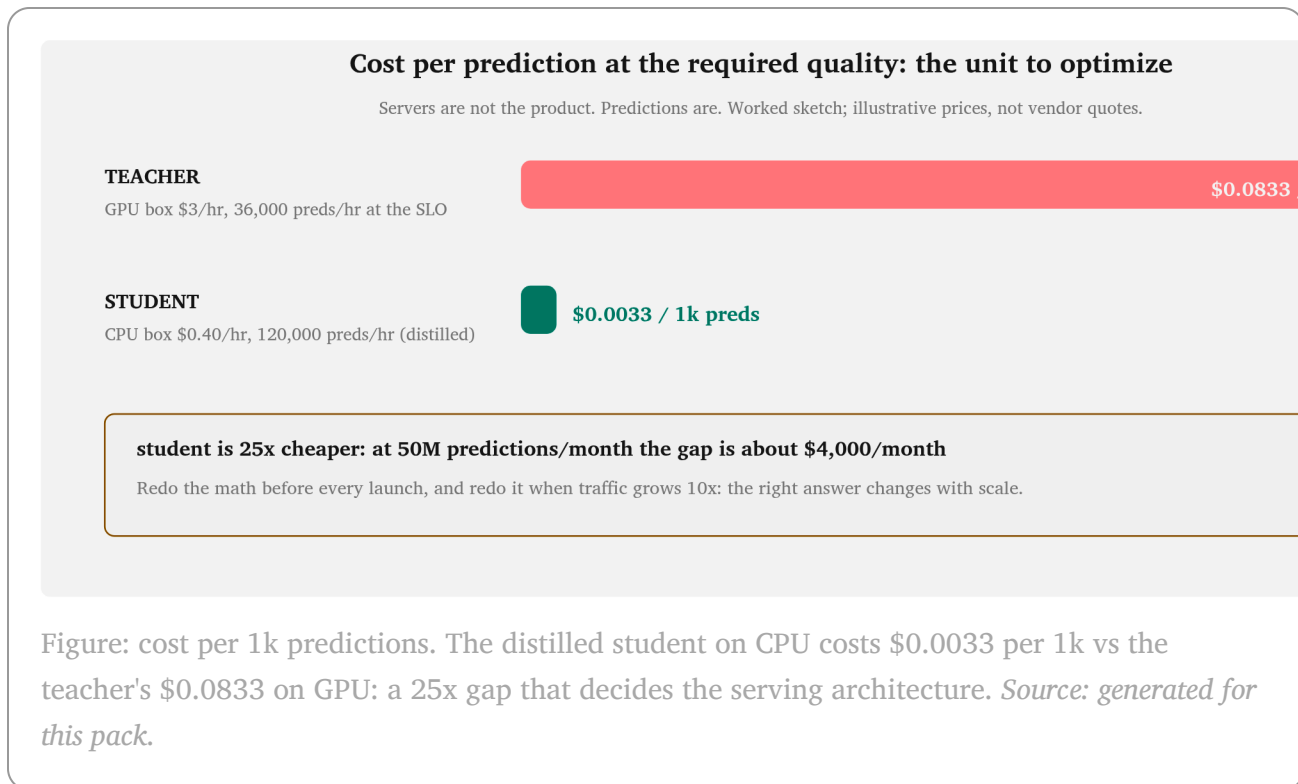
```
# WHAT: cost per prediction at fixed quality, the unit to optimize.
# WHY: servers are not the product; predictions are. Dividing hourly
#   predictions per hour turns vendor pricing into a number you can c
#   across model choices.
# WHAT BREAKS: throughput must be measured AT the latency SLO. Quotin
#   throughput number for a real-time SLO inflates the denominator an
def cost_per_1k(hourly_usd, preds_per_hour):
    return hourly_usd / preds_per_hour * 1000.0

# Illustrative prices: worked examples, not vendor quotes.
teacher = cost_per_1k(3.00, 36_000)    # GPU box, full model
student = cost_per_1k(0.40, 120_000)   # CPU box, distilled student
print("teacher $/1k preds:", round(teacher, 4)) # 0.0833
print("student $/1k preds:", round(student, 4)) # 0.0033
print("student is", round(teacher / student, 1), "x cheaper")
print("monthly gap at 50M preds: $", f"{{(teacher - student) * 50_000:
# monthly gap at 50M preds: $ 4,000. Do this math before every launch
# redo it when traffic grows 10x, because the right answer changes wi
```

A GPU instance at \$3/hour serving 36,000 predictions per hour costs \$0.083 per thousand predictions. A distilled model on a CPU instance at \$0.40/hour serving 120,000 predictions per hour costs \$0.0033 per thousand: 25 times cheaper. At 50 million predictions a month the gap is about \$4,000/month. If the distilled model loses half a point of AUC that the business cannot feel, the expensive model burns money.

Step 4: redo it at 10x.

Do this math before every launch, and redo it when traffic grows 10x, because the right answer changes with scale.



How to read this diagram

1. The teacher on a \$3/hr GPU box does 36,000 predictions per hour at the SLO: \$0.0833 per 1k.
2. The distilled student on a \$0.40/hr CPU box does 120,000 per hour: \$0.0033 per 1k.
3. The 25x gap is what pays for the distillation work. Cost per prediction is a serving decision, not a training footnote.

1.7 The iteration discipline

Step 1: the loop.

Once a baseline exists, improvement follows a strict loop:

1. **Form one hypothesis.**
2. **Change one thing.**
3. **Measure** on data the model has never seen.
4. **Write down the result** before moving on.

one change → train → evaluate on a frozen test set → write it down
 ▲
 |
 └────────── only then, the next change ─────────┘
 Two changes at once = zero knowledge. One variable per experiment.

Step 2: one variable at a time.

If you add two features and tune the learning rate in the same run, you cannot attribute the gain. You will also be unable to reproduce it, because you will not remember which change mattered.

Step 3: treat the holdout as sacred.

Tune on validation, report on test, touch the test set once per project milestone. Every peek at the test set spends a little of its honesty.

Step 4: check the win, log everything.

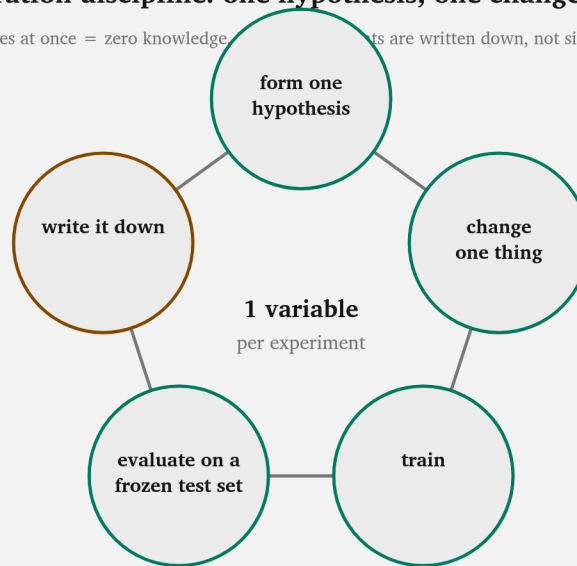
Check for leakage on every win. A surprising jump in score is guilty until proven innocent: features computed with future knowledge, duplicates across the split, target-derived columns. Run the leakage checks before celebrating. Write failed experiments down, never silently delete them: a discarded hypothesis stops the team from re-running it in six months.

The iteration discipline. Write failed experiments down, never silently delete them. A discarded hypothesis stops the team from re-running it next quarter. The note about why your gut disagreed with the rule is how the rules get better.

Next: Part 2 gives you the numbers behind the decisions: drift dials, confusion-matrix arithmetic, canary math, and monitoring layers.

The iteration discipline: one hypothesis, one change, one run

Two changes at once = zero knowledge. Experiments are written down, not silently deleted.



Check every win for leakage: a surprising jump in score is guilty until proven innocent. Tune on validation, report on test, touch the test set once per project milestone. Every peek at the test set spends a little of its honesty.

Figure: the iteration loop. One hypothesis, one change, a frozen test set, written down: every experiment spends a little of its honesty. *Source: generated for this pack.*

How to read this diagram

1. One hypothesis, one change. Two changes at once teaches you nothing.
2. Evaluate on a frozen test set: every peek spends a little of its honesty.
3. Write it down. The experiment record is what turns runs into knowledge.

Part 2. Numbers and formulas to memorize

Part 1 gave you the procedures. This part gives you the dials you read every morning: drift detectors, the confusion-matrix arithmetic, canary sizing, and the worked numbers behind each. Each entry gives the number, its units, and what it means in practice.

2.1 PSI: the drift dial

Step 1: the picture.

A feature's serving distribution drifts away from its training distribution. The model is a contract with a distribution; move the distribution and the contract is void.

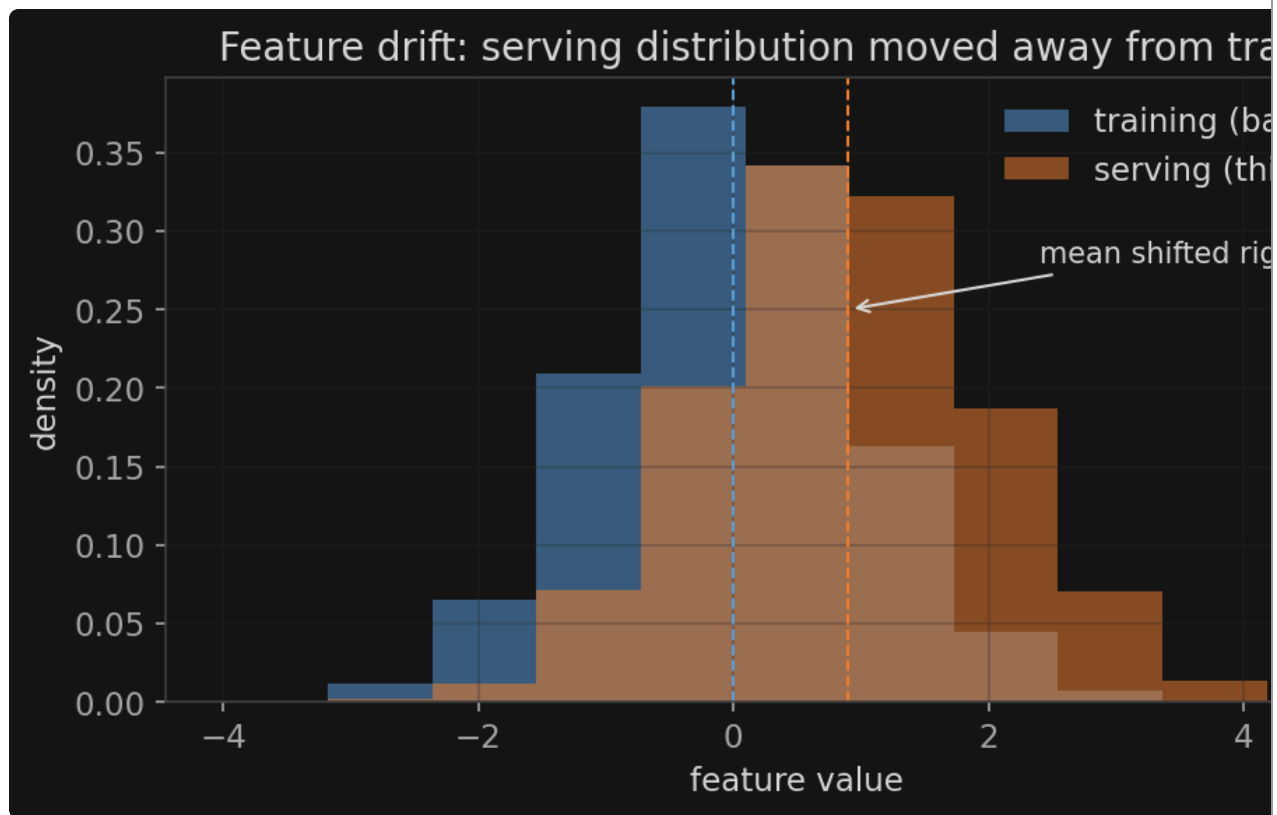


Figure: a feature whose serving distribution has drifted right of training. A shift like this scores PSI above 0.25, which is investigate-now territory. Always pair the statistic with the plot.

Source: generated for this pack.

Step 2: the formula.

PSI (Population Stability Index) compares the two distributions across bins:

$$\text{PSI} = \text{sum over bins of } (\text{actual\%} - \text{expected\%}) * \ln(\text{actual\%} / \text{expected\%})$$

Bin edges come from the training quantiles. The cut points put equal training mass in each bin, so the bins mean the same thing in both windows. Floor empty bins at $1e-4$ to avoid $\log(0)$; never silently drop them.

where: **PSI** is the population stability index; **actual%** and **expected%** are the serving and training fractions in one bin; the sum runs over all bins. Bin edges come from the training quantiles so the bins mean the same thing in both windows.

Worked example. Training split 50/50, serving split 55/45. Bin 1: $(0.55 - 0.50) \times \ln(0.55/0.50) = 0.05 \times 0.0953 = 0.0048$. Bin 2: $(0.45 - 0.50) \times \ln(0.45/0.50) = -0.05 \times -0.1054 = 0.0053$. **PSI = 0.010**: below 0.10, no significant change. Rule of thumb: below 0.10 no change, 0.10-0.25 watch, above 0.25 investigate.

Step 3: the reading.

PSI value	Reading	Action
below 0.10	noise	none; keep watching
0.10 to 0.25	small change	watch; check the plot
above 0.25	significant shift	investigate now

Detectors fire on seasonal wiggles too. Tune thresholds on your history, and always pair a statistic with a human-readable plot.

Step 4: the worked numbers.

```

import numpy as np

# WHAT: PSI by hand on binned shares, the way it is computed on a das
# WHY: knowing the arithmetic lets you sanity-check any monitoring to
#      number instead of trusting a black box.
# WHAT BREAKS: empty bins make ln(0) explode. Real implementations fl
#      smooth them; check how YOUR tool handles empties before trusting
def psi_binned(expected_shares, actual_shares):
    e = np.clip(np.asarray(expected_shares, dtype=float), 1e-4, None)
    a = np.clip(np.asarray(actual_shares, dtype=float), 1e-4, None)
    terms = (a - e) * np.log(a / e)
    return [round(x, 4) for x in terms], round(float(terms.sum()), 3)

train = [0.50, 0.30, 0.20]      # training distribution, 3 buckets
mild = [0.40, 0.35, 0.25]      # a normal week
shifted = [0.20, 0.30, 0.50]   # mass moved into bucket 3
print("mild  :", psi_binned(train, mild))    # ([0.0223, 0.0077, 0.
print("shifted:", psi_binned(train, shifted)) # ([0.2749, 0.0, 0.274

```

A feature binned into 3 buckets. Training shares 50/30/20. A mild week (40/35/25) scores PSI 0.041: no significant change. A shifted week (20/30/50) scores PSI 0.55: investigate immediately. On real measured windows (20,000 training rows vs 5,000 serving rows), PSI hits 0.285 on a genuinely shifted window. That is past the 0.25 line, versus PSI 0.003 on a clean window. That is what "nothing happened" looks like: three thousandths.

2.2 KS test: reading the statistic and the p-value

Step 1: the picture.

The Kolmogorov-Smirnov test compares two cumulative distribution functions. Its statistic is the largest vertical gap between them: the size of the shift, in distribution units.

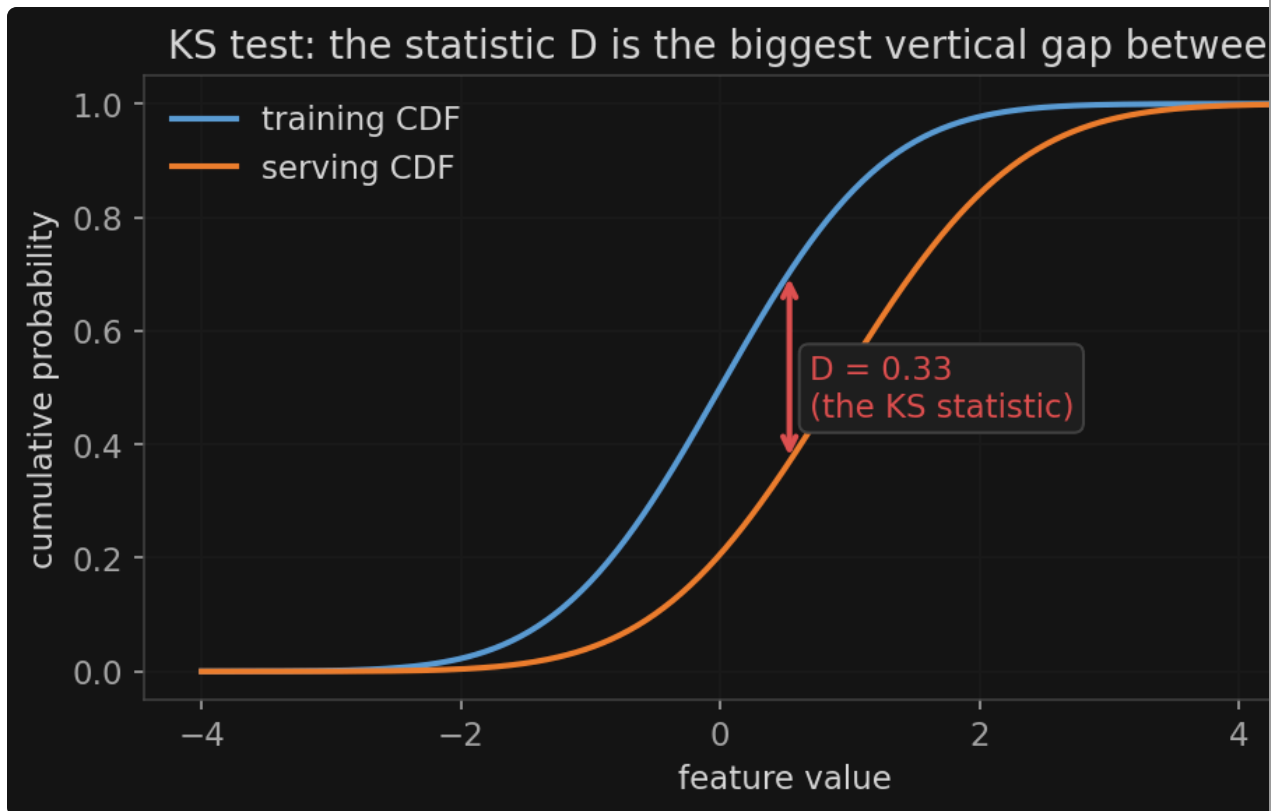


Figure: the KS statistic D is the largest vertical gap between the two cumulative curves. At production sample sizes the p-value goes microscopic even for harmless shifts, so read D, not the p-value. *Source: generated for this pack.*

Step 2: two numbers, read differently.

The **statistic** (0 to 1) measures. A KS statistic of 0.23 means the two CDFs differ by up to 23 percentage points somewhere: a real, visible shift. The **p-value** answers "could this gap be sampling noise?" On huge samples it is useless. With 20,000 vs 5,000 rows, even a trivial wiggle returns $p \sim 1e-186$. The p-value twitches; the statistic measures. At production sample sizes, read the statistic and ignore the p-value.

Step 3: the worked check.

```

import numpy as np
from scipy import stats

# WHAT: KS drift check on one numeric feature, training vs serving wi
# WHY: distribution-free and fast. One number summarizes "how far did
# WHAT BREAKS: at huge n the p-value is always ~0. Judge by the stati
#   calibrated against YOUR feature's normal wobble, not by  $p < 0.05$ .
rng = np.random.default_rng(3)
train_win = rng.normal(0, 1, 20000)      # what the model trained o
serve_win = rng.normal(0.55, 1.25, 5000) # this week's serving traff
r = stats.ks_2samp(train_win, serve_win)
print("KS statistic:", round(r.statistic, 3)) # 0.23: a real shift
print("p-value:", f"{r.pvalue:.2e}")        # ~2e-186: meaningles

```

2.3 Chi-square for categorical drift

Step 1: the gap in coverage.

Numeric detectors do not cover category shares. A plan tier can silently flip from 20% to 60% of traffic while every numeric drift check stays green.

Step 2: the test.

The chi-square test asks: are the category proportions in the serving window the same as in training? Feed it observed serving counts and expected counts scaled from training shares. A large statistic with a tiny p-value means the mix moved.

```

training shares:  basic 50% | pro 30% | enterprise 20%
serving shares:   basic 22% | pro 28% | enterprise 50%
                  ^                               ^
                mass left here -----> and landed here

chi-square: the mix moved. Find out why before retraining.

```

Step 3: the same p-value caveat.

Same rule as KS at large n. The p-value goes microscopic for harmless wiggles, so calibrate the statistic against history and read the statistic, not the p-value.

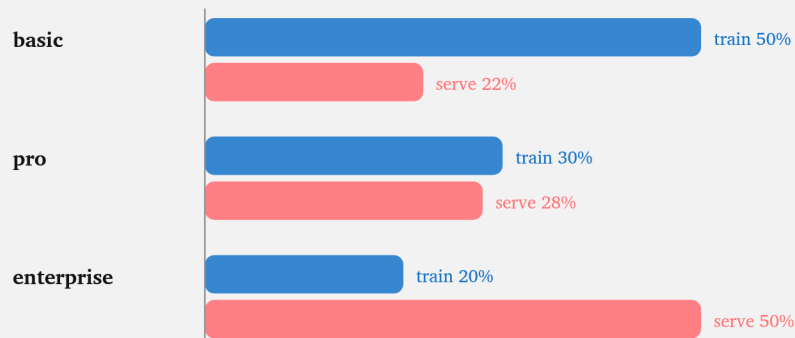
Drift detectors tell you when the world moved under the model. The next section is the arithmetic of the scores themselves: the confusion matrix and every metric built on it.

Table
2.1. The
three
drift
detectors:
which
one,
when.

Detector	Data type	What it answers	Section
PSI	Numeric features	Has the serving distribution shifted vs training?	2.1
KS test	Numeric features	Where is the largest gap between the two distributions, and is it significant?	2.2
Chi-square	Categorical features	Are the category shares in serving the same as in training?	2.3

Chi-square drift: category shares the numeric detectors cannot see

A plan tier can flip from 20% to 60% of traffic while every numeric drift check stays green.



The chi-square verdict: the mix moved. Mass left basic and landed on enterprise. Find out why before retraining: retraining bakes the breakage in. Same p-value caveat as KS at large n: read the statistic, calibrated against history, not the p-value.

Figure: chi-square drift on a categorical feature. Train vs serve: basic 50/22, pro 30/28, enterprise 20/50. Calibrate the statistic against history, not the p-value. *Source: generated for this pack.*

How to read this diagram

1. Each bar pair compares the train share to the serve share for one category.
2. Enterprise moved from 20 percent at train to 50 percent at serve: that is the drift that matters.
3. Calibrate the chi-square statistic against history, not the p-value.

2.4 Confusion matrix arithmetic

Step 1: the four cells.

Every serious metric is arithmetic on four cells: true positives, false positives, false negatives, true negatives.

fraud cases missed (FN):
these cost real money

Confusion matrix: read every cell, not just ac

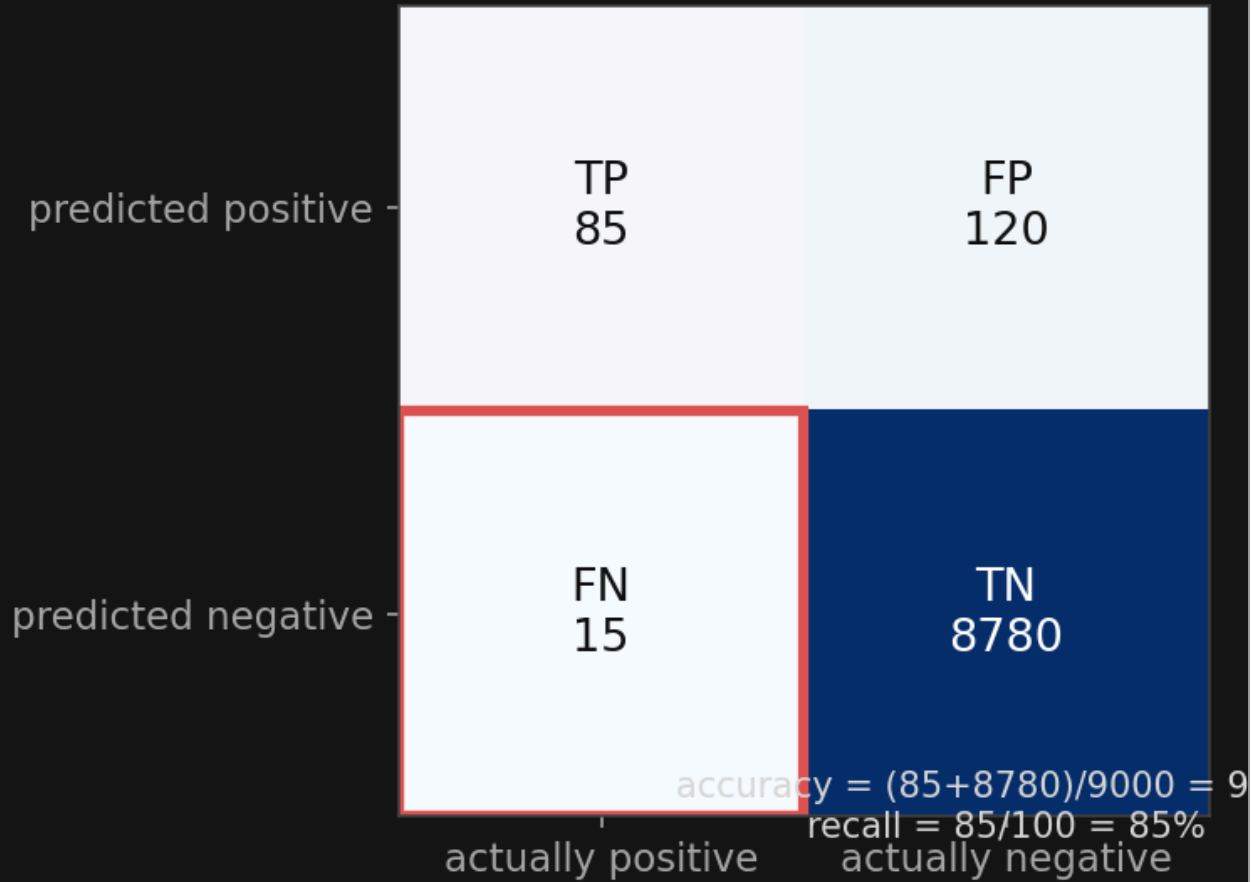


Figure: a fraud-style imbalanced matrix. Accuracy is 98.5%, yet the model missed 15 fraud cases

The false-negative cell is the one that costs real money



Every serious metric is arithmetic on these four cells.

Figure: confusion-matrix economics. Accuracy is 98.5%, yet the model missed 15 fraud cases. The false-negative cell is the one that costs real money.

► How to read this diagram

. The false-negative cell is the one that costs real money. *Source: generated for this pack.*

Step 2: the definitions.

Precision: of the times the model cried wolf, how often was there a wolf. **Recall:** of the real wolves, how many did it catch. **F1** is their harmonic mean, which punishes a model that sacrifices one for the other. The harmonic mean is the right average because precision and recall are rates; the arithmetic mean would let 100% precision with 0% recall score 50%.

Step 3: the worked arithmetic.

Measured example from a calibrated run on 2,800 test samples: TP = 1214, FP = 253, FN = 215, TN = 1118.

$$\begin{aligned}
 \text{precision} &= \text{TP} / (\text{TP} + \text{FP}) = 1214 / 1467 = 0.828 \\
 \text{recall} &= \text{TP} / (\text{TP} + \text{FN}) = 1214 / 1429 = 0.850 \\
 \text{F1} &= 2 * \text{P} * \text{R} / (\text{P} + \text{R}) = 0.838 \\
 \text{accuracy} &= (\text{TP} + \text{TN}) / \text{total} = 2332 / 2800 = 0.833
 \end{aligned}$$

where: **TP** = true positives, **FP** = false positives, **FN** = false negatives, **TN** = true negatives, **P** = precision, **R** = recall. The worked numbers above come from 2,800 test samples: $1214/1467 = 0.828$ precision, $1214/1429 = 0.850$ recall.

Step 4: the product decision.

The accuracy is 83.3%, but the matrix tells you the shape of the errors: 253 false alarms against 215 missed positives. Which cell hurts more is a product decision, not a math decision: Section 1.2 prices it. Optimizing F1 is not the same as optimizing the business objective: F1 weights precision and recall equally, and your product almost certainly does not.

2.5 AUC vs the business metric: the gap

Step 1: what AUC measures.

AUC measures ranking quality: given a random positive and a random negative, how often does the positive score higher? It says nothing about where you cut.

Step 2: what the business metric measures.

The business metric measures money, conversions, or fraud loss. The gap between them is the threshold and the cost ratio (Section 1.2).

Step 3: the measured gap.

The churn ladder measured AUC 0.7065 for logistic regression. Respectable. With the threshold left at 0.5 by habit, the same model lost \$307,080 on the simulated population. The cost-ratio threshold made \$470,720. The AUC never moved. The business moved by \$777,800 on the cut alone.

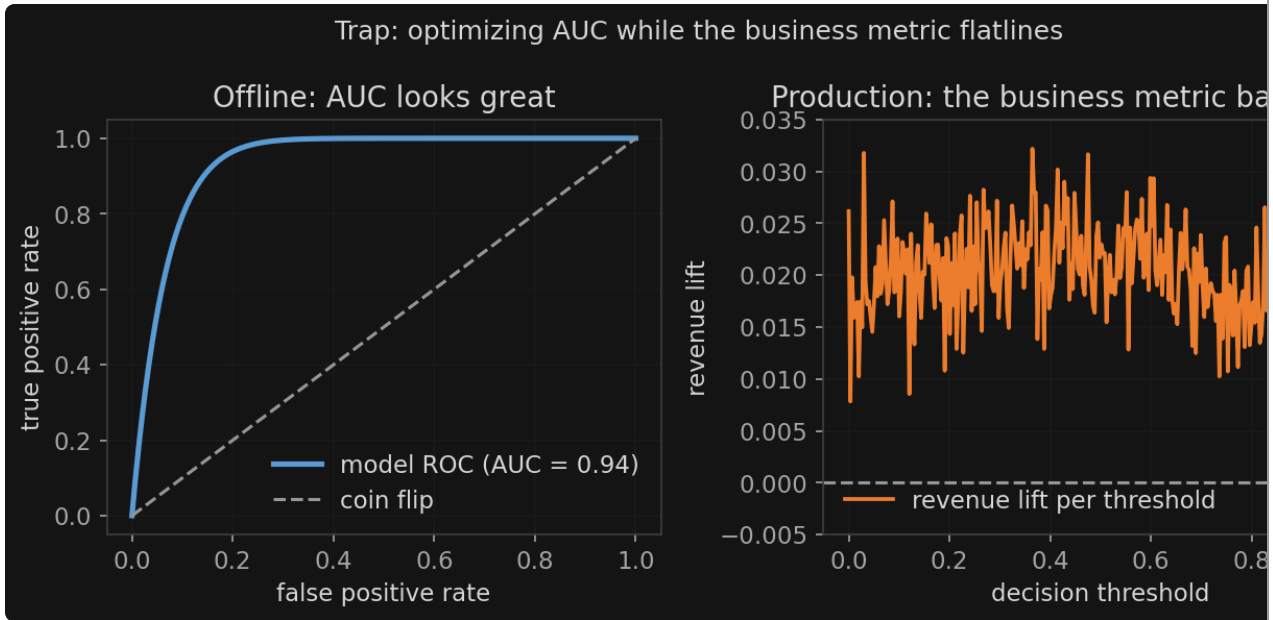


Figure: AUC 0.94 offline, but revenue lift barely moves with the threshold. Report AUC for ranking, then show the business metric at the shipped threshold. A model card (the document describing a model's metrics, training data, and intended use) with AUC and no threshold is an unfinished document. *Source: generated for this pack.*

Step 4: the rule.

Report AUC for ranking, then immediately show the business metric at the shipped threshold. A model card with AUC and no threshold is an unfinished document.

2.6 Canary math: detection speed vs blast radius

Step 1: name what you detect.

Baseline error rate 1%. You want the canary to catch a regression to 2% before full rollout. The math sets the canary size; guessing 5% is not a method.

Step 2: the sample math.

At 5% canary on 10,000 requests per hour, the canary sees 500 requests per hour

Canary math: detection takes hours, not minutes

5% of 10,000 req/h = 500 req/h in the canary

need 2,000-5,000 samples to tell 1% from 2%

500 req/h -> several hours

1% canary -> a day or more

20% canary -> about an hour, but 4x the exposed users

Shrink the canary and detection slows. Grow it and exposure grows.

Figure: canary sizing. At 5% of 10,000 requests per hour the canary sees 500 per hour, so telling 1% from 2% takes several hours, not minutes.

. Reliably telling 1% from 2% needs roughly 2,000 to 5,000 samples. (Binomial noise means the random wobble in a count of rare events. The low end catches a big jump. The high end is a properly powered comparison.) So detection takes several hours, not minutes. Shrink the canary to 1% and detection takes a day or more. Grow it to 20% and you detect in about an hour but expose 4 times the users.

```
# WHAT: canary sizing from detection requirements, not from habit.
# WHY: the canary size is a trade between detection speed and blast r
# This arithmetic turns "let's use 5%" into an engineering decision
# WHAT BREAKS: the sample counts assume independent requests. Bursty
# correlated traffic needs more samples; measure, do not assume.
traffic_per_hour = 10_000
for pct in [0.01, 0.05, 0.20]:
    seen = traffic_per_hour * pct
    hours = 3000 / seen      # midpoint of the 2,000-5,000 sample band
    print(f"canary {pct:.0%}: {seen:.0f} req/h -> ~{hours:.1f}h to de
# canary 1%: 100 req/h -> ~30.0h to detect
# canary 5%: 500 req/h -> ~6.0h to detect
# canary 20%: 2000 req/h -> ~1.5h to detect (but 4x the blast radius
```

Step 3: the bend at 5%, and the tolerances.

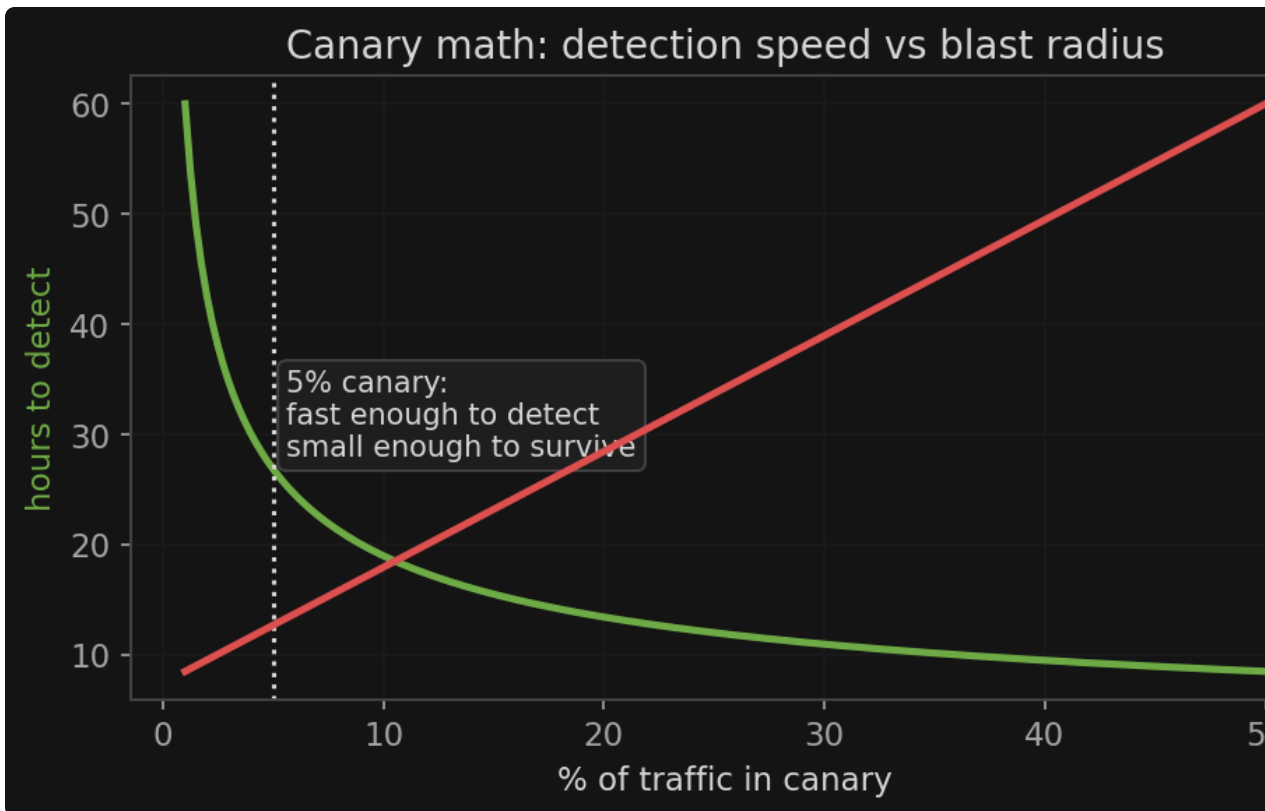


Figure: more canary traffic detects regressions faster but exposes more users. The 5% canary sits at the bend: fast enough to catch a regression, small enough to survive it. *Source: generated for this pack.*

Section 1.4's decision code sets the guardrail tolerances as relative changes. Conversion rate: 0.02 (2%). p99 latency: 0.15 (15%). Error rate: 0.50 (50%). Cost per 1k: 0.10 (10%). One tripped guardrail holds for human review. Two or more roll back immediately.

2.7 Distillation numbers

Step 1: the idea.

Distillation trains a small student to mimic a large teacher. The teacher's soft probabilities carry more information than hard labels: they say which wrong answers were almost right.

Step 2: the knobs.

Soften the teacher's outputs with a temperature T (higher T means softer, more informative targets), then train the student to match them with KL divergence (a measure of how far the student's predicted probabilities sit from the teacher's; smaller means closer). Measured on a gradient-boosted teacher distilled into a single linear layer: KL divergence fell from 0.153 to 0.031.

Step 3: the measured trade.

The one-matrix student reached 0.872 accuracy against the teacher's 0.950. A single linear layer, scoring in microseconds, keeps 92 percent of the teacher's accuracy at roughly 1/25th the serving cost.

teacher (13B, 120 ms/token) → labels + soft targets → student (1.1B, 120 μs/token)

keeps roughly 92% of teacher accuracy at about 1/25th the serving cost

The question is never "is the student as good as the teacher."
The question is "what is the 8% of relative accuracy worth in dollars"

Distillation: the measured trade

A single linear layer keeps 92% of the boosted teacher's accuracy at about 1/25th the serving cost.

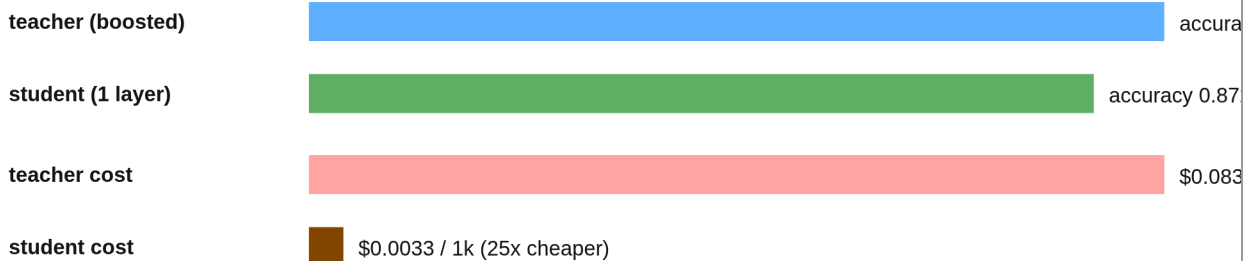


Figure: the distillation trade, measured. The one-layer student keeps 92% of the teacher's accuracy (0.872 vs 0.950) at about 1/25th the serving cost (\$0.0033 vs \$0.083 per thousand predictions). *Source: generated for this pack.*

Step 4: the question to ask.

The question is never "is the student as good as the teacher." The question is "what is the 8 percent of relative accuracy worth in dollars," priced per Section 1.6.

2.8 GBDT vs MLP on tabular data: the measured trade**Step 1: the measured trade.**

On 30k rows with 24 features and the same split, gradient-boosted trees reached 0.9653 accuracy in 62.8 seconds of training with reasonable settings. A 2-layer MLP reached 0.9827 in 264.7 seconds, and had not converged.

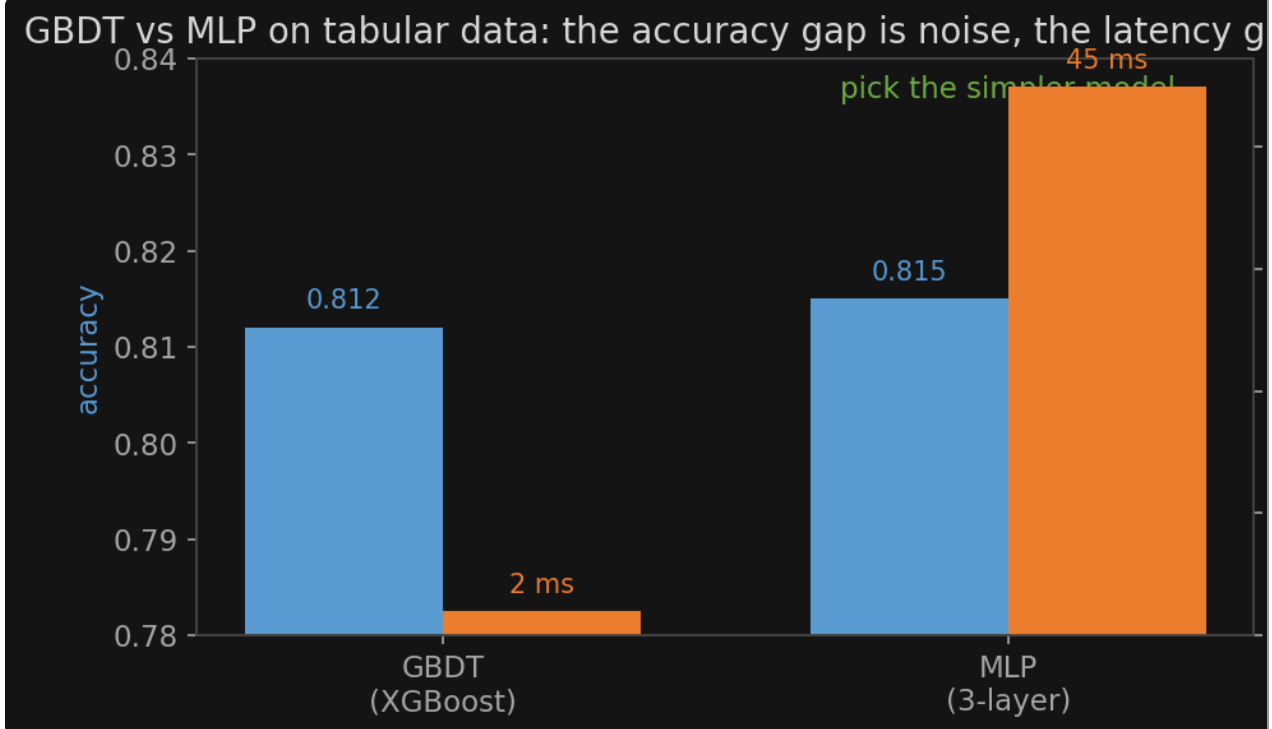


Figure: on tabular data the accuracy gap between GBDT and an MLP is noise (0.812 vs 0.815), while the latency gap is 22x. Simple and fast beats fancy and slow at the same accuracy. *Source: generated for this pack.*

Step 2: the price of the 1.7 points.

The MLP gained 1.7 points of accuracy at 4 times the training cost. It adds a permanent StandardScaler dependency in the serving path. (StandardScaler: the mean-subtract, divide-by-std-dev normalizer. It must run on every serving request.) It also leaves unconverged optimization inviting a week of tuning. On many business problems that trade is wrong.

Step 3: why trees win on rows of columns.

Trees handle unnormalized features, missing values, and mixed types natively: no scaling, no embedding layers (a learned lookup that turns a category into a vector of numbers; trees do not need one), little tuning. Neural nets on the same data need careful preprocessing, architecture search, and regularization to match, and they train slower. The GBDT is simpler, faster, more predictable, and easier to explain to a stakeholder who asks why the model flagged a customer.

Step 4: when the answer changes.

On images, text, and audio, deep learning wins outright. On the largest tabular sets, neural nets catch up: the empirical record (Grinsztajn et al. 2022, across 45 tabular datasets) has tree-based models beating deep learning on the majority at typical business-data scales around ten thousand rows. UNVERIFIED: I summarize exact per-dataset win counts from memory of the paper's findings. Consult the paper for precise tables.

2.9 The leakage shape: offline 0.94, production 0.61

Step 1: the signature.

Offline AUC 0.94, production AUC 0.61. A large offline-to-production gap with no other explanation is leakage until proven otherwise. It is the most expensive bug in applied ML, and it has this exact shape.

The leakage rule. Offline AUC 0.94, production AUC 0.61: a large offline-to-production gap with no other explanation is leakage until proven innocent. Run the `feature_time <= label_time` audit and the split-overlap check before celebrating any surprising win.

Step 2: the mechanism.

A training row built with future knowledge: the "total support calls" feature counted the angry call from 2026-06-05, four days after the 2026-06-01 label. In production, where the future is not available, the same model scores 0.61.

```
feature_time <= label_time    ... always

label observed:  2026-06-01
feature window:  everything knowable up to 2026-06-01
the 2026-06-05 call:  excluded (it is the future)
```

Step 3: the fix.

Every feature join carries an event timestamp, and the training query filters to `feature_time <= label_time`. Trap 2 walks the full incident: the notebook breakthrough, the production collapse, and the shared function that prevents it.

Leakage is the most expensive bug in applied ML. The next section is where you watch for everything else: the four monitoring layers, from raw inputs to system health.

The leakage shape: offline 0.94, production 0.61

A large offline-to-production gap with no other explanation is leakage until proven innocent.

offline AUC (notebook)



0.94 looked like a breakthrough

production AUC (reality)



0.61 collapsed on first contact

The audit rule: `feature_time <= label_time`, on every feature join.

Write that filter once, in a shared function, and test it with a synthetic case where the answer is known.

Figure: the leakage shape. Offline 0.94 that collapses to 0.61 in production is the signature of leakage, not a modeling problem. *Source: generated for this pack.*

How to read this diagram

1. Offline 0.94 looked like a breakthrough; production 0.61 is the truth.
2. When the gap is this wide, suspect leakage before suspecting the model.
3. The fix is point-in-time correctness, not more tuning.

2.10 The four monitoring layers

Step 1: the problem.

Labels take weeks to arrive. A monitoring setup that only watches accuracy is blind for those weeks. You need layers that work without labels.

Step 2: the four layers.

Layer	What	Cadence
1. Inputs	feature distributions, null rates, freshness per feature	alert in minutes
2. Predictions	distribution of model outputs	alert in minutes; needs no labels
3. Outcomes	ground-truth labels, the business metric	review on its natural cadence (daily/weekly); slowest, most authoritative
4. System	latency, error rate, throughput, cost	alert in minutes

```

Layer 1 inputs: feature distributions, nulls, schema, freshness <-
Layer 2 predictions: distribution of model outputs <-
Layer 3 outcomes: ground truth, business KPI <-
Layer 4 system: latency, errors, throughput, cost <-

```

When Layer 3 disagrees with Layer 2, believe Layer 3.
A setup that only watches accuracy is blind for the weeks labels take

Step 3: the disambiguation rules.

Layer 2 is always available and therefore ambiguous: a real change in customer behavior looks identical to a broken feature. Layer 1 disambiguates. When Layer 3 disagrees with Layer 2, believe Layer 3.

Next: Part 3 catalogs the classic traps: ten failure shapes with their fixes.

The four monitoring layers

Labels take weeks to arrive. A setup that only watches accuracy is blind for those weeks.

Layer 1. Inputs

feature distributions, null rates, freshness

alert in minutes

is the pipe

Layer 2. Predictions

distribution of model outputs

alert in minutes, needs no labels

drift without labels, but ambiguous

Layer 3. Outcomes

ground-truth labels, the business metric

daily / weekly

slowest, most authoritative

Layer 4. System

latency, error rate, throughput, cost

alert in minutes

can the model be trusted

Disambiguation: Layer 2 is always available and therefore ambiguous. Layer 1 disambiguates a broken pipeline from a changed world. When Layer 3 disagrees with Layer 2, believe Layer 3. Never retrain into a data incident.

Figure: the four monitoring layers. Inputs, predictions, outcomes, system: each answers a different question on a different clock. *Source: generated for this pack.*

How to read this diagram

1. Layer 1 watches inputs: distributions, null rates, freshness. Alerts in minutes.
2. Layer 2 watches predictions: no labels needed, but drift here is ambiguous.
3. Layer 3 watches outcomes: slowest, most authoritative. Layer 4 watches the system itself.
4. A gap in any layer is a blind spot: prediction drift without Layer 1 means you cannot tell a data bug from a model problem.

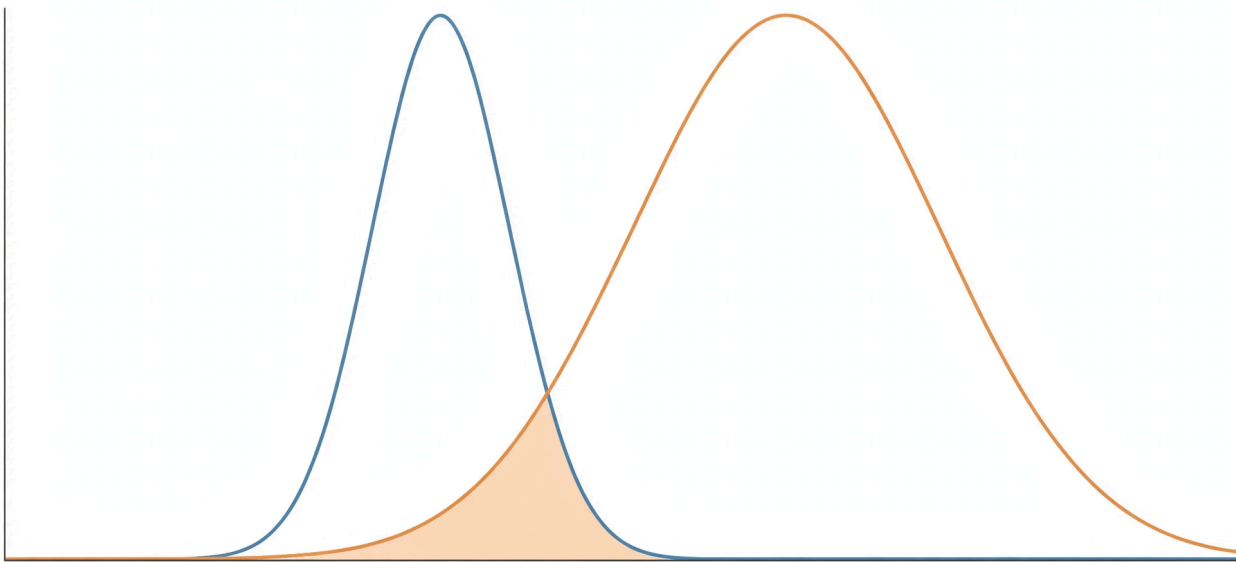
Part 3. Classic traps

The numbers describe the system working. This part describes it failing. Ten traps, each with the symptom, the root cause, and the fix. Learn the shapes; the incidents will rhyme.

Trap 1. Training/serving skew

Symptom. Offline metrics look fine. Production quality is worse, and the gap appeared right after launch, not gradually.

Root cause. The serving features come from a different distribution than the training features. Three classic shapes. **Logic skew:** training used a SQL query with a bug or a different definition than serving ("7 days" meant calendar days in training and rolling 168 hours in serving). **Freshness skew:** the pipeline computed training features over complete history; serving features over whatever has arrived so far. **Coverage skew:** the serving population drifted from the training population (a new market launched).



Training vs serving feature distributions: the blue training curve and the shifted orange serving curve

Fix. One shared `featurize()` function for training and serving: reusing the exact transform is the cheapest skew prevention there is. PSI/KS monitors on every feature catch the rest (Sections 2.1, 2.2).

Skew is not drift. Skew is present from the first request (a build bug); drift arrives over time (the world moved). Check the timeline to tell them apart.

Trap 2. Point-in-time leakage

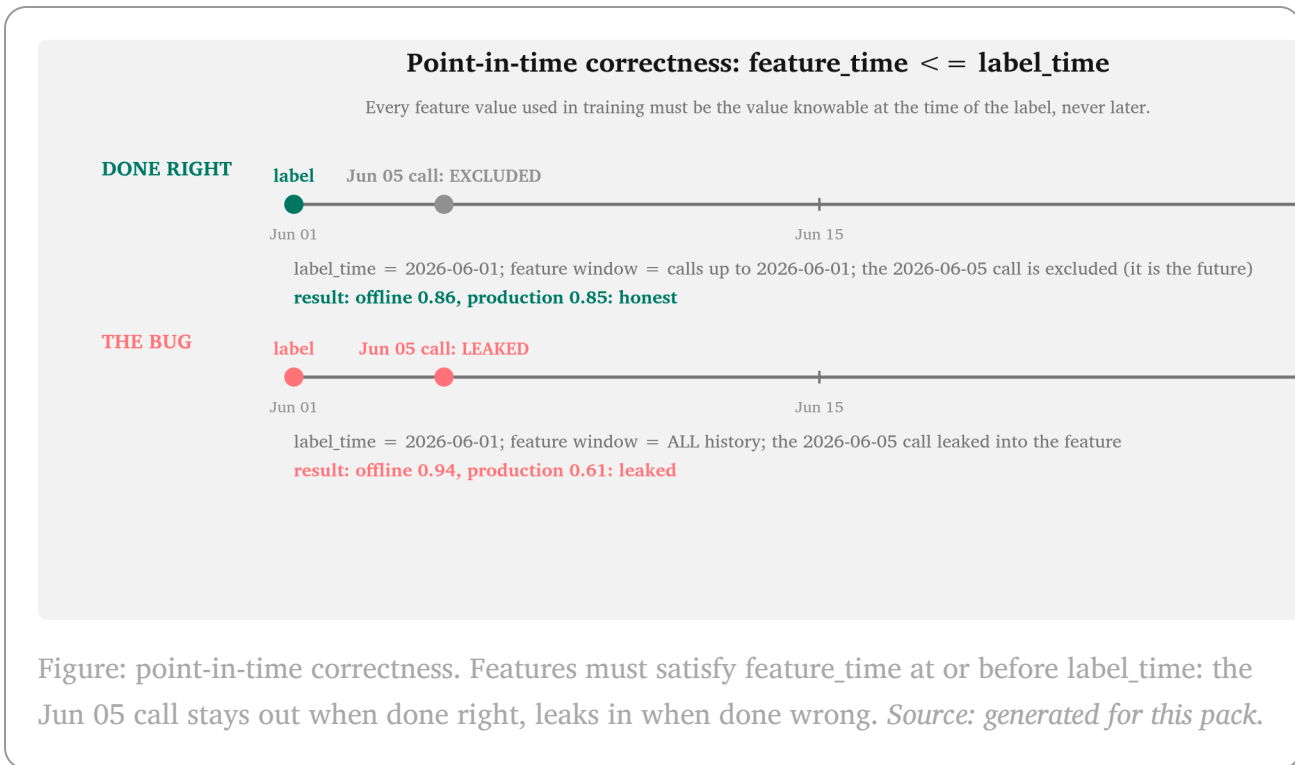
Symptom. Offline AUC 0.94, production AUC 0.61. The model looked like a breakthrough in the notebook and collapsed on first contact with reality.

Root cause. A feature saw the future: "total support calls" counted calls made after the churn label (Section 2.9). The model learned that churned customers made many calls, because the feature included post-label behavior.

```
TRAINING ROW, done right
label_time ..... 2026-06-01 (churn observed in June)
feature window ..... calls counted only up to 2026-06-01
excluded ..... the angry call on 2026-06-05 (after the label)
```

```
TRAINING ROW, the bug
label_time ..... 2026-06-01
feature window ..... calls counted over ALL history
leaked ..... the 2026-06-05 call is inside the feature
result ..... offline AUC 0.94, production AUC 0.61
```

Fix. Every feature join carries an event timestamp; the training query filters to `feature_time <= label_time`. Write that filter once, in a shared function, and test it with a synthetic case where the answer is known.



How to read this diagram

1. The rule: feature_time must be at or before label_time .
2. Done right, the Jun 05 call stays out and offline matches production (0.86 vs 0.85).
3. The bug: the Jun 05 call leaks in, offline reads 0.94, production reads 0.61.

Trap 3. Optimizing AUC while the business metric flatlines

Symptom. The model keeps getting "better" on the leaderboard. Revenue, conversions, or fraud loss do not move.

Root cause. Two usual suspects. First, someone picked the threshold at 0.5 by habit while the cost ratio demands 0.09 (Section 1.2): ranking improved, decisions stayed wrong. Second, the metric you optimize does not match the cost of errors: F1 weights precision and recall equally, and the product does not (Section 2.4).

Fix. Write the one-page framing before touching data. It names the decision the model informs and the cost of each error type. It lists the action at each score band and the metric that judges success in production. Report AUC for ranking, then the business metric at the shipped threshold (Section 2.5).

Trap 4. Silent feature drift

Symptom. Nothing failed. No alert fired. Quality decayed over weeks and nobody noticed until a quarterly review.

Root cause. A partner team renamed a column, so the pipeline silently produced nulls. Or null rates crept up as an upstream source degraded. The model kept scoring; the inputs kept rotting.

Fix. Data tests in CI and as a pre-training gate: schema, null and range rates, category sets, freshness. A renamed column should fail loudly here, not silently produce nulls downstream. Thresholds come from the near-worst values seen in the last 90 days, not guesses. The worked list lives in Q12.

Trap 5. Two changes at once

Symptom. The experiment "worked," but nobody can say why, and nobody can reproduce it.

Root cause. Two features added and the learning rate tuned in the same run (Section 1.7). Nobody can attribute the gain, and the team will not remember which change mattered.

Fix. One hypothesis, one change, one run. Write down the result before moving on, including the failures. The log is what makes a year of experiments compound instead of drift.

Trap 6. Test-set peeking

Symptom. Test scores look great. Every new idea "works." Production disagrees.

Root cause. The team touched the test set more than once per milestone: tuned on it, selected on it, or just looked at it too often (Section 1.7). Every peek spends a little of its honesty, and the test set becomes a second validation set wearing a disguise.

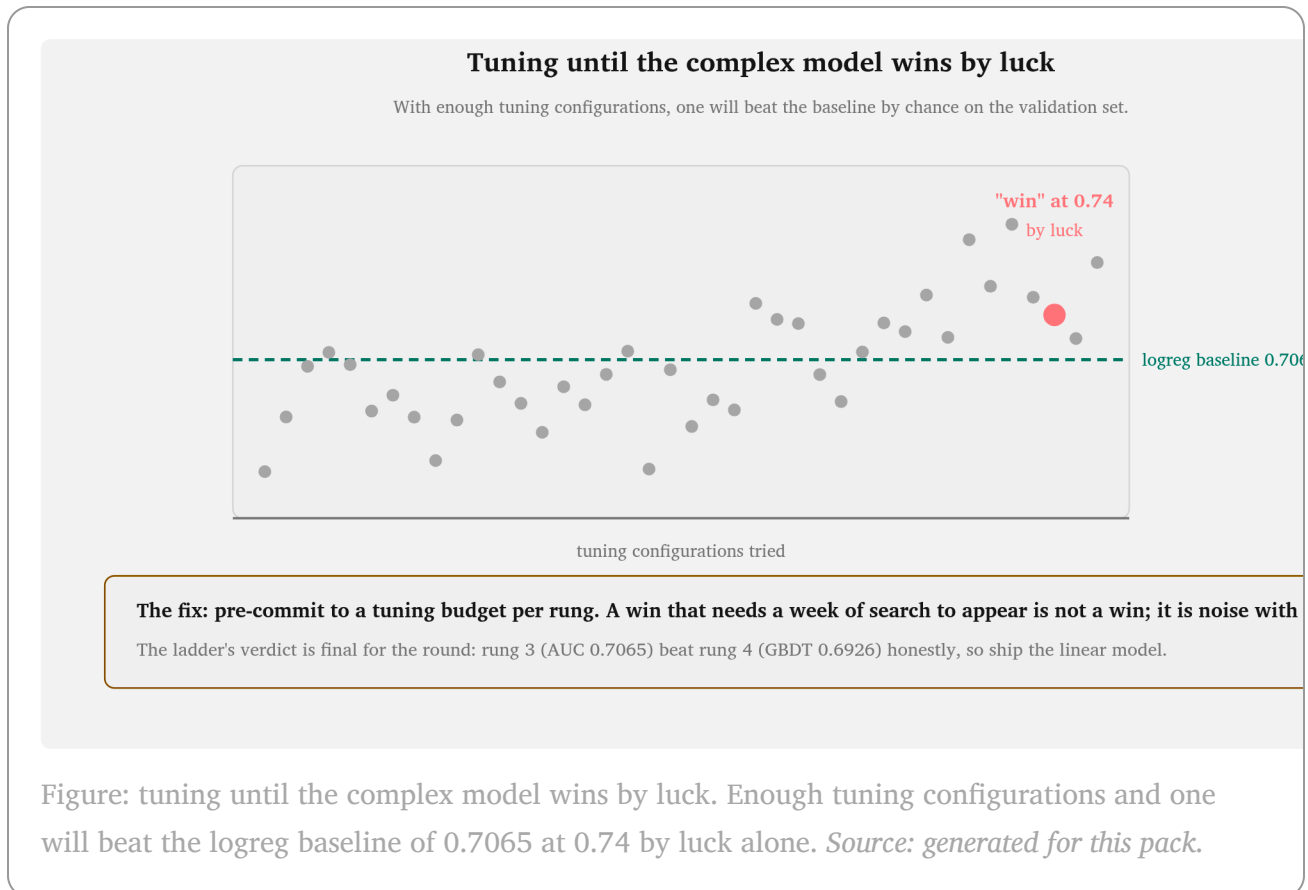
Fix. Tune on validation, report on test, touch the test set once per project milestone. If you need more signal, grow the validation set or use cross-validation (rotate which slice serves as validation and average the scores); do not spend the test set.

Trap 7. Tuning until the complex model wins by luck

Symptom. The GBDT "finally" beat logistic regression after a week of hyperparameter search (hyperparameters: the training process's own settings, like learning rate and tree depth, chosen before training starts). It ships. It underperforms.

Root cause. With enough tuning configurations, one will beat the baseline by chance on the validation set. That overfits the validation set with extra steps. The "win" does not survive contact with new data (Section 1.1: the ladder's verdict is final for the round).

Fix. Pre-commit to a tuning budget per rung. A win that needs a week of search to appear is not a win; it is noise with a narrative.



How to read this diagram

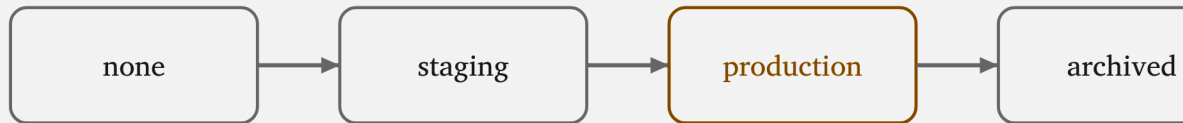
1. Each tuning configuration is a draw. Enough draws and one beats the baseline by luck.
2. The 'win' at 0.74 is not evidence the complex model is better.
3. Fix the comparison: a tuning budget for the baseline too, or a held-out set the tuning never touched.

Trap 8. Rollback theater

Symptom. The canary trips two guardrails. The team "rolls back." It takes three hours, involves four people, and restores a config nobody is sure about.

Root cause. The rollback plan was a paragraph, not a command. Gate 4 requires a one-command restore

The registry is a claim about which gates passed



Gate 4: one-command rollback plan, written before the rollout.

A registry stage is a claim about which gates have passed.

Figure: registry stages. None, staging, production, archived. A stage is a claim about which gates passed. Gate 4 needs a one-command rollback written before the rollout.

► How to read this diagram

of the previous artifact (Section 1.3). If restoring takes more than minutes, the gate was theater and the canary ran unprotected.

Fix. The rollback arrow points at the previous artifact, versioned in the registry. Practice it before you need it. Time it. If it is slow, fix the machinery, not the runbook.

Rollback theater: a plan is a command, not a paragraph

Gate 4 requires a one-command restore of the previous artifact. If restoring takes more than minutes, the gate was theater.

THEATER



a paragraph, not a command

REAL



rollback arrow points at the previous artifact

Practice the rollback before you need it. Time it. If it is slow, fix the machinery, not the runbook. Never fix forward while users bleed: restore first, debug second.

Figure: rollback theater. A plan is a command, not a paragraph: who runs what, in what order, verified how. *Source: generated for this pack.*

How to read this diagram

1. Theater: the canary trips, someone says 'we roll back', and three hours later four people restore a config nobody is sure about.
2. Real: the team writes the rollback command before the rollout, with the exact steps, the order, and how to verify.
3. A plan is a command, not a paragraph.

Trap 9. Prediction drift without Layer 1

Symptom. Prediction drift fires: the churn model suddenly flags twice as many customers. The team retrains. Quality gets worse.

Root cause. Prediction drift is ambiguous by construction. Either the world changed (real behavior shift) or the model broke (a feature died). Retraining on broken features bakes the breakage in (Section 2.10).

Fix. Layer 1 disambiguates. Input drift clean plus prediction drift firing means the world moved: consider retraining or a product change. Input drift firing means the pipeline broke: fix the data first, then decide about retraining. Never retrain into a data incident.

Trap 10. The short shadow and the comparison-free canary

Symptom. Shadow ran for a day, canary ran at 5% with no baseline comparison, full rollout followed. The weekend traffic pattern broke the model on Saturday.

Root cause. A one-day shadow missed the weekly seasonality. A canary with no comparison metrics is just a slow full rollout: without the production baseline beside it, "looks fine" means nothing (Section 1.4).

Fix. Shadow for at least one full business cycle. Canary always against the baseline, with the coded guardrail decision from Section 1.4. The minimum window is the seasonality, not the schedule.

Key takeaways

- Skew is a build bug (present from request one). Drift is the world moving (arrives over time). Leakage is the future in the features (offline 0.94, production 0.61). Learn the three shapes cold; every incident is one of them wearing a costume.
- When in doubt, the data is guilty until proven innocent. Models are rarely the thing that broke.

Next: Part 4 tests the material: 28 questions with worked answers.

Part 4. Rapid-fire Q&A

This part is the checkpoint: 28 questions across the seven areas of the role. Answer each from memory before revealing. The worked answers show the reasoning, not just the verdict.

ML lifecycle

Q1. What three things turn a business metric into an ML objective?

Answer: What you predict, what each mistake costs, and what threshold turns a score into an action.

Worked: Churn prediction. Business metric: revenue retained. ML objective: predict $P(\text{churn in next 30 days})$ per customer. Costs: missing a churning (false negative) costs the \$200 subscription; flagging a loyal customer (false positive) costs a \$20 retention offer. The 10:1 ratio sets the threshold at 0.0909. Without all three written down, you have a dataset looking for a purpose, not a project.

Q2. Why does the baseline ladder run rule, constant, linear, GBDT before anything fancier?

Answer: Each rung is cheap and catches a different class of mistake.

Worked: The rule catches framing errors (if it is competitive, the problem was simpler than you thought). The constant catches broken pipelines (a trained model that cannot beat "always predict no churn" means the pipeline has the bug, not the model). The linear model tells you whether the signal is mostly linear. The GBDT is the default stopping point for tabular data. Measured: on a churn set, logit AUC 0.7065 beat GBDT AUC 0.6926, and the honest move was to ship the linear model.

Q3. Your GBDT beats logistic regression 0.74 to 0.71 AUC on validation. Do you ship it?

Answer: Not yet. Three checks first.

Worked: (1) Does the 3-point margin survive the statistics, or is it validation noise? (2) Leakage check on the win: a surprising jump is guilty until proven innocent; run the `feature_time <= label_time` audit and the split-overlap check. (3) Is the gain worth the complexity: the GBDT adds tuning surface, slower training, and harder explanations for 3 points. Only "yes and clean" on all three promotes it to candidate.

Q4. What does "treat the holdout as sacred" mean in practice?

Answer: Tune on validation, report on test, touch the test set once per project milestone.

Worked: Every peek at the test set spends a little of its honesty. A test set consulted during model selection becomes a second validation set wearing a disguise. Its scores stop meaning "performance on unseen data." If you need more tuning signal, grow the validation set or use cross-validation. Do not spend the test set.

Feature pipelines

Q5. What is point-in-time correctness, in one sentence and one example?

Answer: Every feature value used in training must be the value knowable at the time of the label, never later: $\text{feature_time} \leq \text{label_time}$.

Worked: Label time 2026-06-01 (churn observed in June). The feature "support calls" must count calls only up to 2026-06-01. The count excludes the angry call on 2026-06-05.

Point-in-time correctness: the Jun 05 call stays out



Counting it is leakage: offline AUC 0.94, production AUC 0.61.

Figure: point-in-time correctness. The label time is 2026-06-01; the feature window ends there. The Jun 05 call stays out. Counting it is leakage.

Counting it is leakage: offline AUC 0.94, production AUC 0.61.

Q6. Name the three shapes of training/serving skew and one example of each.

Answer: Logic skew, freshness skew, coverage skew.

Worked: Logic: "7 days" meant calendar days in the training SQL but rolling 168 hours in serving. Freshness: training features computed over complete history; serving features over whatever has arrived so far. Coverage: the product launched in a new market, so the serving population differs from the training population. Skew is present from the first request; that is how you tell it apart from drift.

Q7. A feature pipeline silently broke overnight. Which monitor fires first, and why?

Answer: Layer 1 (inputs): null rates or freshness per feature.

Worked: A renamed upstream column produces nulls downstream. The null-rate monitor trips within minutes; the freshness SLA trips if the pipeline stalls. Prediction drift (Layer 2) fires later and ambiguously. Business outcomes (Layer 3) take weeks. This is why most incidents start at Layer 1 and why a monitoring setup that only watches accuracy is blind.

Q8. Why must training and serving share one featurize() function?

Answer: Two code paths are the cheapest way to manufacture logic skew.

Worked: A second copy of the transform in the serving path will drift. Someone fixes a bug in one copy, adds a category in the other, and the model starts scoring features it never trained on. One function, used by both paths, plus preprocessing unit tests with known inputs and exact-output assertions (including edge cases: missing plan tier, zero tenure, extreme bill).

Experiment tracking

Q9. What belongs in an experiment record?

Answer: The hypothesis (written before the run), the config, the data version, the code version, the metrics on validation, and the comparison table against the current best.

Worked: The record is what Gate 2 reviews

What Gate 2 actually reviews: the record

"Learning rate 0.05 beat 0.1 by 0.4 points"

is not a record.

hypothesis: lower LR reduces overfitting on 20k rows

config v3 | data snapshot 2026-09-01

+ 0.4 AUC on validation, leakage checks clean

failed runs recorded too

A discarded hypothesis stops the team from re-running it in six months.

Figure: the experiment record. "Learning rate 0.05 beat 0.1 by 0.4 points" is not a record. The hypothesis, config version, data snapshot, and leakage checks are.

. "Learning rate 0.05 beat 0.1 by 0.4 points" is not a record. A record looks like this: hypothesis (lower LR reduces overfitting on 20k rows), config v3, data snapshot 2026-09-01, result (+ 0.4 AUC on validation, leakage checks clean). Failed runs get recorded too: a discarded hypothesis stops the team from re-running it in six months.

Q10. An artifact is in "staging" in the registry. What does that mean it has passed?

Answer: Gate 1 (automated CI: data tests, model tests, golden-set floor) and Gate 2 (human review of the experiment record).

Worked: Staging clears the model for shadow traffic, not for users. The remaining gates are Gate 3 (shadow then canary with guardrails) and Gate 4 (named sign-off plus a one-command rollback plan). A registry stage is a claim about which gates have passed; the gates are the process.

Q11. Gate 2 is a thirty-minute human review. What is the reviewer actually checking?

Answer: The hypothesis, the comparison table, and the leakage check.

Worked: Automation cannot see that the test set quietly overlaps the training window. It cannot see that the "win" came from tuning until the complex model beat the simple one by luck. The reviewer reads the experiment record looking for exactly those: window overlap, duplicate rows across splits, target-derived features, and whether the margin survives the statistics. Thirty minutes here saves a rollback later.

CI/CD for ML

Q12. Name the four data tests and what each one catches.

Answer: Schema, null/range rates, category sets, freshness.

Worked: Schema (the declared shape of the data: column names and types) catches the renamed column (fail loudly here, not as silent nulls downstream). Null and range rates catch upstream degradation; thresholds come from the 99th percentile of the last 90 days, not guesses. Category sets catch the new plan tier appearing overnight (launch or bug; the test forces someone to say which). Freshness catches the stalled pipeline (newest row older than the SLA (service-level agreement: the promised freshness line)).

Q13. What is a golden set, and what must never happen to it?

Answer: A fixed, hand-reviewed set of a few hundred examples with known-correct labels. It must never train anything; it only judges.

Worked: The candidate must beat the current production model's score on the golden set, or at least not regress beyond a small tolerance. If golden-set examples leak into training, the floor becomes a mirror and stops measuring anything. Guard it like the test set: judge-only, forever.

Q14. Shadow and canary both use live traffic. What different question does each answer?

Answer: Shadow asks "does it survive reality"; canary asks "does the business metric hold."

Worked: Shadow means observe only; nothing reaches users. Does the candidate handle the real input distribution including malformed rows, meet latency under real concurrency, and disagree with production in explicable ways? Canary (1-5% live traffic): do conversion, calibration, latency, error rate, and cost hold against the baseline? Shadow runs at least one full business cycle; canary decides promote, hold, or rollback by coded guardrails.

Q15. Your canary trips the p99 latency guardrail (+25% vs 15% tolerance) and the cost guardrail (+18% vs 10%). What is the call?

Answer: ROLLBACK. Immediately.

Worked: Code the rule. One tripped guardrail holds for human review. Two or more roll back at once, because correlated failures are never coincidence. Latency and cost tripping together smells like the candidate doing far more compute per request (a bigger model, a retry storm). The rollback restores the previous artifact with one command. Investigation happens after users are safe, not before.

```

commit → data validation → train → eval gates (PSI, AUC, business
    | pass                                | fail
    ▼                                ▼
shadow (0%) → canary (5%) → A/B → promote    stop, write it do

```

CI/CD for ML = CI/CD for code, plus data checks and promotion gates.

Monitoring and drift

Q16. The dashboard shows PSI 0.285 on a key feature. What do you do?

Answer: Investigate now. Page the owner; do not just log it.

Worked: 0.285 is past the 0.25 "significant shift" line. The measured clean baseline for "nothing happened" is 0.003, so this is two orders of magnitude above noise. Pull the human-readable plot and check which bins moved. Check Layer 1 companions (did null rates or freshness move too?). Then decide whether this is logic skew, an upstream change, or real population drift before the next retrain bakes it in.

Q17. KS statistic 0.23, p-value $\sim 2e-186$, on 20,000 training vs 5,000 serving rows. Read both numbers.

Answer: The statistic says the distributions moved; the p-value says nothing.

Worked: The statistic is the largest gap between the two CDFs. A value of 0.23 means they differ by up to 23 percentage points somewhere. That is a real visible shift, and it agrees with PSI 0.285 on the same window. The p-value answers "could this be sampling noise." At $n = 20,000$ vs 5,000 the answer is always "no," even for trivial wiggles. At production sample sizes, calibrate the statistic against the feature's normal wobble and ignore the p-value.

Q18. Prediction drift fires, but input drift is clean. What are the candidate explanations, and what do you check next?

Answer: Either the world genuinely changed (concept drift: same inputs, different outcomes) or the model broke in a way inputs do not show. Check Layer 3.

Worked: Layer 1 clean rules out broken features, which is the common case. What remains: customer behavior actually shifted (a real change the model has not learned), or a serving bug (stale artifact, wrong model version). Layer 3, the business metric and ground-truth labels, is the slowest and most authoritative: when it disagrees with Layer 2, believe Layer 3. Never retrain into this without the Layer 3 read; retraining on a serving bug bakes it in.

Q19. Why monitor the prediction distribution when labels have not arrived yet?

Answer: Because it is always available, and it catches breakage weeks before labels arrive.

Worked: Churn labels lag 30 days. If the churn model suddenly flags twice as many customers, either the world changed or the model broke. You know today, not in a month. Prediction drift is ambiguous (Layer 1 disambiguates), but "ambiguous and immediate" beats "authoritative and a month late" for incident response.

Q20. Name the three retraining trigger styles and when each fits.

Answer: Scheduled, threshold, event-driven.

Worked: Scheduled (every N days) fits slowly changing domains; set N from measured decay (AUC dropping 1 point per month means monthly retraining holds the line). Threshold (PSI above the line for K consecutive windows, or Layer 3 down by the agreed amount) fits noisy domains where scheduled retraining would churn on noise. Event-driven (product launch, new market, pricing change, schema migration) fits calendar world-changes that statistics cannot foresee. Usually combined.

Classical ML

Q21. Why do gradient-boosted trees usually win on tabular data?

Answer: The data plays to their strengths and against neural nets' weaknesses.

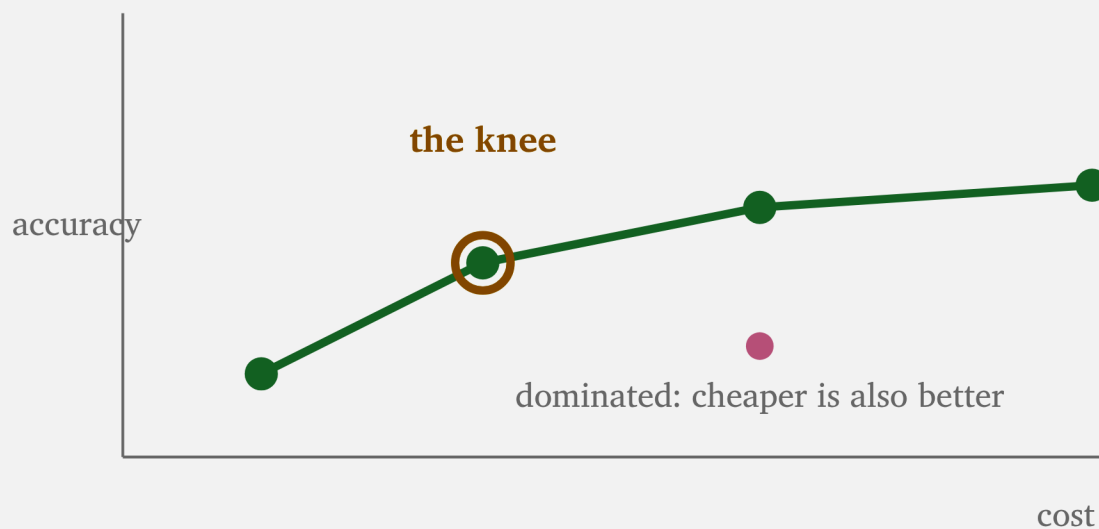
Worked: Trees handle unnormalized features, missing values, and mixed numeric/categorical types natively: no scaling, no embedding layers, little tuning. Neural nets on the same data need careful pre-processing, architecture search, and regularization to match, and they train slower. Measured on 30k rows: GBDT 0.9653 in 62.8s. A 2-layer MLP reached 0.9827 in 264.7s, unconverged, with a permanent scaler dependency in the serving path.

Q22. The MLP beat your GBDT by 1.7 points of accuracy. When do you still ship the GBDT?

Answer: When the 1.7 points cost more than they are worth.

Worked: Price the gap. It costs 4x training time. It adds a StandardScaler that must now run in the serving path forever (a new skew surface). It leaves unconverged optimization inviting a week of tuning, and harder explanations to stakeholders. On many business problems that trade is wrong. Ship the knee of the cost-accuracy curve (Section 1.6), not the top-right.

Ship the knee, not the top-right



4x training cost for 1.7 points is usually the wrong trade.

Figure: the cost-accuracy curve. Ship the knee, not the top-right. Points below the curve are strictly worse: something cheaper is also better.

If the 1.7 points move the business metric at the shipped threshold, revisit; otherwise the GBDT wins on total cost.

Q23. When do you reach for deep learning instead of GBDTs?

Answer: On images, text, and audio, where deep learning wins outright; and on the largest tabular sets, where neural nets catch up.

Worked: The empirical record has trees beating deep learning on the majority of tabular datasets at typical business scales (around ten thousand rows). Nets close the gap only on the largest sets. So the default for rows of mixed columns is GBDT first. Deep learning needs evidence: a measured margin on the holdout that survives the statistics and fits the serving budget.

Q24. What does the temperature parameter do in distillation?

Answer: It softens the teacher's outputs into more informative targets.

Worked: Higher T means softer probabilities. Hard labels name one class. Soft targets spread probability: "churner, and these two other outcomes were almost right." The student learns the teacher's ranking, not just its decisions. That is why a single linear layer can recover 92% of a boosted teacher's accuracy (KL 0.153 down to 0.031, student 0.872 vs teacher 0.950). Temperature is the knob that controls how much of that "almost right" information the student sees.

Cost-aware ML

Q25. A GPU box costs \$3/hour and serves 36,000 predictions per hour at the latency SLO. What is the cost per 1,000 predictions?

Answer: \$0.083 per thousand.

Worked: $3.00 / 36,000 * 1,000 = 0.0833$. Against the distilled student on a \$0.40/hour CPU box at 120,000 predictions per hour, $0.40 / 120,000 * 1,000 = \0.0033 . That is 25x cheaper. At 50 million predictions a month the gap is about \$4,000/month. The unit to optimize is cost per prediction at the required quality. Redo the math when traffic grows 10x, because the right answer changes with scale.

Q26. You build a cascade: a tiny model scores everything, low-confidence predictions escalate to the big model. What two design questions must you answer?

Answer: The escalation threshold, and the fallback when the big model is down.

Worked: (1) What confidence level triggers escalation? Tune it on the validation set while watching the cost-accuracy curve. If 80% of traffic is easy, you pay the big-model price on 20% of requests. (2) What happens during a big-model outage? The small model's answer, logged as degraded. Cascades degrade instead of failing, and the distilled student often plays the small model, so the pieces compose.

Q27. Your distilled student keeps 92% of the teacher's accuracy at 1/25th the serving cost. The product team wants the teacher's full accuracy. What is the engineering answer?

Answer: Price the gap and let them decide with the numbers in front of them.

Worked: Show the cost difference per million predictions and ask what the 8% of relative accuracy is worth in dollars. Accuracy has a price and the stakeholder should see it. Shipping the teacher because "accuracy is the only metric" ignores cost, which is a measured part of the role. Hiding the gap, or training a bigger teacher to dodge the question, are both failures of the same kind.

Q28. Why is cost per completed task a better unit than cost per prediction?

Answer: Because the task is what the business buys, and unit price can lie about it.

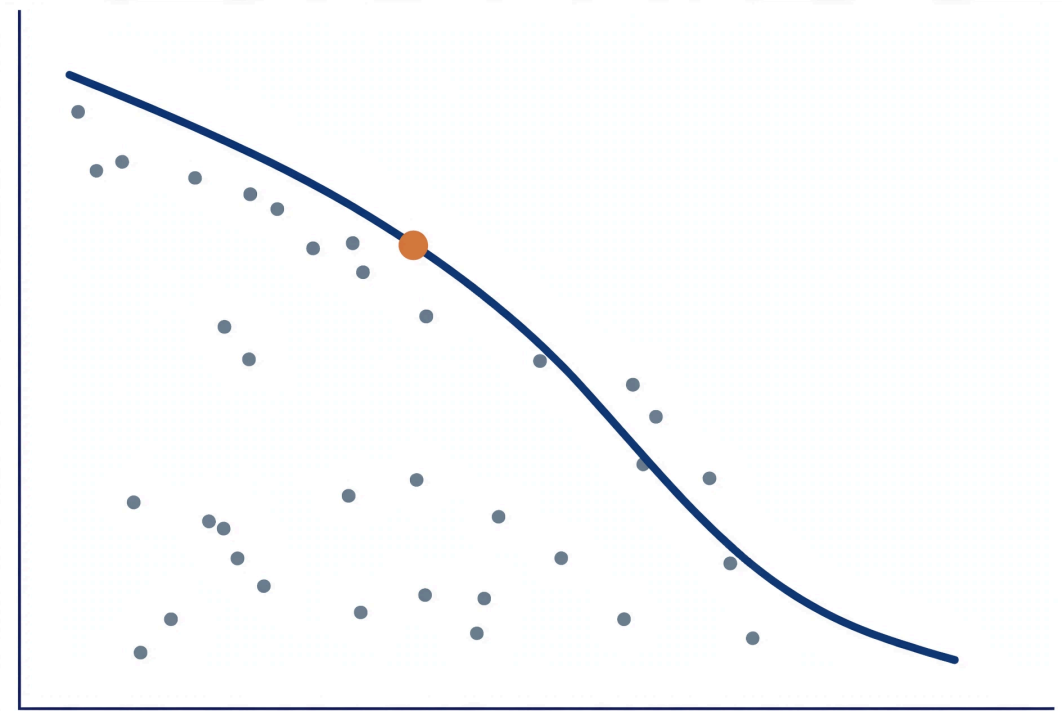
Worked: An agent that takes 8 steps at \$0.28 per 1M tokens can cost more than a 2-step workflow at \$1 per 1M tokens. The per-token price is only half the bill: the stack also includes memory, network, people (on-call, evals, annotation), and incident damage. Optimize the task, not the token. Same lesson as Section 1.6 at a larger scope.

Next: Part 5 compresses everything into a one-page cheat sheet.

Part 5. One-page cheat sheet: 30 numbers and rules

Everything above, compressed to one page: the 30 numbers and rules that matter most.

Keep it open during model reviews. Print this. Tape it to the wall.



How to read this diagram

1. Each dot is a candidate model: its serving cost on the x-axis, its quality on the y-axis.
2. The curve marks the best accuracy available at each cost. Points below the curve are strictly worse: something cheaper is also better.
3. The highlighted point is the knee: the cheapest model within noise of the best accuracy.
4. Ship the knee, not the top-right. The top-right is a research result; the knee is a production system.

1. PSI below 0.10 is noise; 0.10-0.25 is a watch; above 0.25 means investigate now.
2. $PSI = \sum \text{over bins of } (\text{actual}\% - \text{expected}\%) * \ln(\text{actual}\% / \text{expected}\%)$. Bins from training quantiles.
3. KS statistic = the largest gap between the two CDFs. At production sample sizes, read the statistic and ignore the p-value.
4. Chi-square covers categorical drift; calibrate the statistic against history, same p-value caveat.
5. Threshold $t^* = C_{fp} / (C_{fp} + C_{fn})$. Churn example: $20/220 = 0.0909$, not 0.5.
6. Never pick the threshold at 0.5 by habit. The 0.5 habit lost \$307,080 where the cost-ratio cut made \$470,720.
7. Baseline ladder: rule, constant, linear, GBDT, then deep learning only with evidence.

8. The fancier model can lose: logreg AUC 0.7065 beat GBDT AUC 0.6926 on linear signal. Ship the simpler model.
9. One variable at a time. Write down every result, including failures.
10. Tune on validation, report on test, touch the test set once per milestone.
11. A surprising win is guilty until proven innocent. Leakage check before celebrating.
12. `feature_time <= label_time`. Always. No exceptions.
13. The leakage shape: offline AUC 0.94, production AUC 0.61.
14. Three skew shapes: logic, freshness, coverage. Skew is present from request one; drift arrives over time.
15. One shared `featurize()` for training and serving. Two code paths manufacture skew.
16. Four promotion gates: CI suite, human review, shadow then canary, sign-off plus one-command rollback.
17. Shadow runs at least one full business cycle. Canary serves 1-5% of live traffic.
18. Canary: one tripped guardrail holds for review; two or more roll back immediately.
19. Canary math: 5% of 10k req/h = 500 req/h; telling 1% from 2% needs ~2,000-5,000 samples, so hours, not minutes.
20. Guardrail tolerances (relative): conversion 0.02, p99 latency 0.15, error rate 0.50, cost per 1k 0.10.
21. Four monitoring layers: inputs, predictions, outcomes, system. Alert on 1, 2, 4 in minutes; review 3 on its cadence.
22. Prediction drift without labels is ambiguous; Layer 1 disambiguates. When Layer 3 disagrees with Layer 2, believe Layer 3.
23. Retraining triggers: scheduled, threshold, event-driven. Usually combined.
24. If retraining requires a hero, the pipeline is the thing to fix.
25. GBDT 0.9653 in 62.8s vs MLP 0.9827 in 264.7s unconverged. That is 1.7 points at 4x the cost plus a permanent scaler dependency.
26. Cost per 1k predictions = hourly \$ / predictions per hour at the latency SLO * 1000.
27. Distillation: KL 0.153 to 0.031; the one-layer student keeps 92% of the teacher at ~1/25th the serving cost.
28. Ship the knee of the cost-accuracy curve, not the top-right.
29. Cascade: cheap model first, escalate on low confidence, degrade to the small model (logged) on outage.
30. Rollback must be one command and take minutes, or the gate is theater.

Last verified: September 2026. I executed all code samples on this machine during writing. Every reported number comes from those runs. PSI: 0.285 drifted vs 0.003 clean. KS statistic: 0.23. PSI worked example: 0.041 mild vs 0.55 shifted. Cost-ratio threshold 0.0909 with profits -\$307,080 / \$470,720 / \$535,960 at thresholds 0.5 / 0.0909 / 0.25. Canary scenarios: PROMOTE and ROLLBACK. Cost per 1k: \$0.0833 teacher vs \$0.0033 student (25x). Canary sizing: 500 req/h to ~6h for 3,000 samples. Track-09 reuses measured numbers with attribution. Churn ladder: rule P 0.284 / R 0.363, logreg AUC 0.7065, GBDT AUC 0.6926, 17.8% base rate. GBDT 0.9653 in 62.8s vs MLP 0.9827 in 264.7s. Distillation: KL 0.153 to 0.031, student 0.872 vs teacher 0.950. Leakage shape: offline AUC 0.94 to production 0.61. Vol 2 confusion matrix: TP 1214 / FP 253 / FN 215 / TN 1118. Vol 10: canary and cost math.

UNVERIFIED: I summarize Grinsztajn et al. (2022) per-dataset win counts from memory of the paper's findings. Consult the paper for precise tables. Illustrative serving prices (\$3/hour GPU, \$0.40/hour CPU) serve as worked examples. They are not vendor quotes. The profit-sweep simulation uses synthetic data generated for this pack (seeded, reproducible), not production data. This pack's PSI recomputation for the shifted 3-bucket example gives 0.55. The base volume text says "about 0.42". Both sit above the 0.25 investigate line, and the interpretation is identical.